

# Relazione di Progetto: Ottimizzazione di un Filtro FIR in High-Level Synthesis (HLS)

Stefano Scarcelli

## Indice

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 1. Introduzione e Analisi degli Obiettivi .....                                 | 1  |
| 1.1. Descrizione del Progetto .....                                             | 1  |
| 1.2. Panoramica delle Tecniche di Ottimizzazione .....                          | 1  |
| 1.3. Obiettivi di Ottimizzazione .....                                          | 1  |
| 2. Metodologia di Esplorazione e Automazione del Workflow .....                 | 2  |
| 2.1. Creazione Workflow base e Validazione .....                                | 2  |
| 2.2. Struttura del Workflow Automatizzato .....                                 | 2  |
| 2.3. Approccio all'Esplorazione «Greedy + Backtracking» .....                   | 4  |
| 3. Metriche di Valutazione e Trade-off .....                                    | 5  |
| 3.1. Metriche Prestazionali (Performance) .....                                 | 5  |
| 3.2. Metriche Hardware (Area) .....                                             | 5  |
| 3.3. Metriche di Consumo (Power) .....                                          | 5  |
| 3.4. Analisi dei Trade-off .....                                                | 6  |
| 4. Esplorazione dello Spazio di Design .....                                    | 6  |
| 4.1. Analisi dell'Architettura Baseline .....                                   | 6  |
| 4.1.1. Implementazione Logica di Base .....                                     | 6  |
| 4.1.2. Testbench C/C++ Integrata .....                                          | 7  |
| 4.1.3. Disabilitazione delle Ottimizzazioni Automatiche .....                   | 7  |
| 4.1.4. Analisi del Testbench VHDL e dei Constraint di Sistema .....             | 8  |
| 4.1.5. Analisi delle Metriche di Valutazione e Limitazioni .....                | 8  |
| 4.2. Ottimizzazioni Architetturali .....                                        | 10 |
| 4.2.1. Eliminazione dei branching: «Code Hoisting» .....                        | 10 |
| 4.2.1.1. Valutazione dei miglioramenti .....                                    | 11 |
| 4.2.2. Separazione dei flussi: «Loop Fission» .....                             | 13 |
| 4.2.2.1. Valutazione dei miglioramenti .....                                    | 14 |
| 4.3. Ottimizzazioni dei Tipi .....                                              | 16 |
| 4.3.1. Ottimizzazione del Datapath: «Bitwidth Optimization» .....               | 16 |
| 4.3.1.1. Adeguamento dell'Ambiente di Validazione .....                         | 16 |
| 4.3.1.2. Valutazione dei miglioramenti .....                                    | 16 |
| 4.3.2. Ottimizzazione della Memoria di Stato: Libreria «ap_shift_reg» .....     | 18 |
| 4.3.3. Valutazione dei miglioramenti .....                                      | 19 |
| 4.3.4. Esplorazione Combinata: «Loop Fission» con «Bitwidth Optimization» ..... | 21 |
| 4.3.5. Valutazione dei miglioramenti .....                                      | 21 |
| 4.4. Parallelismo .....                                                         | 25 |
| 4.4.1. Parallelismo Spaziale Iniziale: Unrolling dello «shift_loop» .....       | 25 |
| 4.4.1.1. Descrizione dell'Implementazione .....                                 | 25 |
| 4.4.1.2. Valutazione dei miglioramenti .....                                    | 26 |

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| 4.4.2. Risoluzione dei Colli di Bottiglia della Memoria: Completamento con «Array Partitioning» ..... | 28 |
| 4.4.2.1. Descrizione dell'Implementazione .....                                                       | 28 |
| 4.4.2.2. Valutazione dei miglioramenti .....                                                          | 28 |
| 4.4.3. Parallelismo Temporale Iniziale: Pipelining del «mac_loop» .....                               | 31 |
| 4.4.3.1. Descrizione dell'Implementazione .....                                                       | 31 |
| 4.4.4. Valutazione dei miglioramenti .....                                                            | 31 |
| 4.4.5. Convergenza di Strategie: Parallelizzazione Avanzata del Ciclo Aritmetico . .                  | 35 |
| 4.4.5.1. Soluzione 09: Loop Pipelining sul Ciclo MAC con Storage Partizionato .....                   | 36 |
| 4.4.5.1.1. Descrizione dell'Implementazione .....                                                     | 36 |
| 4.4.5.2. Soluzione 10: Loop Unrolling Totale del Ciclo MAC .....                                      | 36 |
| 4.4.5.2.1. Descrizione dell'Implementazione .....                                                     | 36 |
| 4.4.5.3. Soluzione 11: Loop Unrolling Parziale del Ciclo MAC .....                                    | 37 |
| 4.4.5.3.1. Descrizione dell'Implementazione .....                                                     | 37 |
| 4.4.6. Valutazione dei miglioramenti .....                                                            | 37 |
| 4.4.7. Esplorazione Specialistica della Memoria: Concorrenza con «ap_shift_reg» .                     | 43 |
| 4.4.7.1. Soluzione 12: Loop Unrolling Totale su Struttura ap_shift_reg .....                          | 44 |
| 4.4.7.1.1. Descrizione dell'Implementazione .....                                                     | 44 |
| 4.4.7.2. Soluzione 13: Pipelining a Livello di Funzione con ap_shift_reg .....                        | 44 |
| 4.4.7.2.1. Descrizione dell'Implementazione .....                                                     | 44 |
| 4.4.8. Valutazione dei miglioramenti .....                                                            | 45 |
| 4.5. Operation Chaining .....                                                                         | 52 |
| 4.5.1. Scelta delle soluzioni .....                                                                   | 52 |
| 4.5.2. Soluzione 03 — Bitwidth opt (on v01) .....                                                     | 53 |
| 4.5.3. Soluzione 07 — Array Partitioning .....                                                        | 54 |
| 4.5.4. Soluzione 10 — Total Unroll (MAC) .....                                                        | 56 |
| 5. Sintesi dei Risultati e Conclusioni .....                                                          | 57 |
| 5.1. Confronto Aggregato delle Soluzioni .....                                                        | 57 |
| 5.2. Analisi delle Migliori Soluzioni .....                                                           | 57 |
| 5.3. Conclusioni .....                                                                                | 58 |

# 1. Introduzione e Analisi degli Obiettivi

## 1.1. Descrizione del Progetto

Il progetto N°1 richiede l'implementazione di varie tecniche di ottimizzazione per la scrittura di un codice C/C++ sintetizzabile tramite approccio *High-Level Synthesis* (HLS) per piattaforme **FPGA**.

Il caso di studio ricade sull'implementazione di un circuito per un filtro **FIR** (*Finite Impulse Response*), partendo da un codice di base (**baseline**) scritto come un tradizionale algoritmo C/C++ orientato all'esecuzione software. Tale codice è privo di alcun accorgimento relativo alla sua sintesi in hardware, risultando pertanto nella generazione di un circuito **estremamente inottimale**.

Lo scopo è dunque quello di ottimizzare il codice attraverso l'applicazione iterativa di diverse tecniche progettuali, al fine di ottenere un blocco **IP** nettamente superiore in termini di **prestazioni, uso delle risorse e dissipazione di potenza** (e consumo di energia).

## 1.2. Panoramica delle Tecniche di Ottimizzazione

Le tecniche di ottimizzazione esplorate riguardano i principali compromessi e le differenze architetturali insite nella definizione di un codice C/C++ pensato per l'esecuzione sequenziale in software rispetto a uno destinato alla sintesi hardware su **FPGA**. Esse includono:

- **Code Hoisting**: Estrazione delle operazioni non dipendenti dall'indice iterativo all'esterno dei loop, oppure ottimizzazione delle espressioni ripetitive all'interno di *branch* condizionali.
- **Loop Fission**: Separazione («*splitting*») delle operazioni eseguite all'interno di un singolo loop iterativo in **molteplici loop più piccoli**, per isolare le dipendenze logiche e semplificare lo *scheduling*.
- **Data Types Optimization**: Uso di tipi a precisione arbitraria (tramite la libreria `ap_int`) per dimensionare accuratamente le variabili, ottimizzando l'allocazione dei registri e l'hardware dedito alle operazioni aritmetiche nel *datapath*.
- **Operation Chaining**: Modifica del vincolo temporale target (periodo di clock) per permettere la concatenazione logica di più operazioni in un singolo ciclo. Questo riduce i tempi morti tra cicli, la complessità della **FSM** (*Finite State Machine*) e il numero di risorse (principalmente *Flip-Flop*) impiegate per la sincronizzazione del flusso dati.
- **Loop Unrolling**: Lo «srotolamento» (parziale o totale) delle iterazioni presenti nei loop, forzando il *tool* a istanziare repliche fisiche dell'hardware per consentire un'elaborazione con parallelismo spaziale.
- **Pipelining**: Implementazione di un'architettura a **pipeline** per sovrapporre temporalmente l'esecuzione di più iterazioni o operazioni sequenziali, massimizzando il *throughput*.
- **Port Interface**: Gestione delle interfacce di **I/O** del modulo (ad esempio tramite direttive di **Array Partitioning** o protocolli standard come **AXI**), al fine di ottimizzare la banda passante dei dati scambiati con il circuito esterno e conformare in maniera efficiente l'operato della **FSM**.

## 1.3. Obiettivi di Ottimizzazione

Oltre alla pura misurazione delle prestazioni relative alle varie implementazioni, un obiettivo centrale è l'analisi critica dell'impatto di ciascuna tecnica HLS: identificare in quali contesti sia valido applicarle e come interagiscano quando vengono combinate in modo sinergico.

Le analisi prestazionali, di efficienza (area e potenza) e di complessità logica rappresentano i parametri principali di valutazione. Tuttavia, lo studio include anche l'esplorazione strategica di

tecniche applicate su rami di sviluppo che producono soluzioni inizialmente e **apparentemente non migliorative** rispetto alla precedente architettura. In questo modo si evita il rischio metodologico di scartare prematuramente una configurazione «sub-ottimale» che potrebbe invece rivelarsi dominante una volta completata l'esplorazione totale dello spazio di design (es. l'effetto combinato di «srotolamento» della memoria e successiva pipeline).

## 2. Metodologia di Esplorazione e Automazione del Workflow

### 2.1. Creazione Workflow base e Validazione

Per impostare un framework di valutazione rigoroso e scientificamente fondato, è stato identificato e seguito il flusso di progettazione digitale standard basato sulla catena di strumenti Xilinx. Questo workflow si articola su due macro-fasi sequenziali: una prima fase di sintesi ad alto livello (**High-Level Synthesis**) per la traduzione dell'algoritmo C/C++ in microarchitettura **RTL**, e una successiva fase di immissione downstream in **Vivado** per la sintesi logica fisica e l'analisi microarchitetturale post-sintesi. Tale approccio è l'unico in grado di estrarre i report analitici e le metriche fisiche reali (timing reali, consumi e occupazione delle slice) necessari per validare e comparare oggettivamente le diverse soluzioni hardware.

Prima di procedere all'esplorazione massiva dello spazio di design, questo intero flusso di lavoro è stato testato e validato eseguendo ogni singolo passaggio in modo completamente **manuale** sulla soluzione di partenza (**baseline**) attraverso l'utilizzo delle interfacce grafiche (**GUI**). Questa fase preliminare manuale si è rivelata fondamentale per mappare l'origine dei dati all'interno dei file di report, comprendere le interdipendenze tra i tool, isolare i vincoli di progetto ottimali e verificare la corretta catena di simulazione (dal comportamento funzionale in **csim** alla verifica temporale in **cosim** e alla generazione dei file di **toggle rate .saif** per l'analisi di potenza della sintesi *Out-Of-Context* (**OOO**) del circuito).

Una volta compreso a fondo il meccanismo e consolidata la stabilità del processo di estrazione sulla **baseline**, l'intera sequenza logica dei passaggi manuali è stata formalizzata e strutturata. L'esperienza accumulata sull'architettura di riferimento ha permesso di definire le specifiche per la creazione degli script automatici, azzerando i tempi morti e i margini d'errore umano nelle successive e numerose fasi di ottimizzazione. Successivamente, questa prima implementazione puramente sequenziale è stata sintetizzata utilizzando la **GUI** di *Vivado HLS*. Questo passaggio esplorativo manuale è stato fondamentale per stabilire il «punto zero» architetturale: estrarre le metriche iniziali di *Latenza*, utilizzo delle risorse logiche e stima temporale del *critical path*, ponendo le basi numeriche per misurare il guadagno (*speed-up*) e l'efficienza delle successive iterazioni di design.

### 2.2. Struttura del Workflow Automatizzato

Per gestire l'elevata mole di varianti architetturali insita nella *Design Space Exploration* (DSE) e garantire l'estrazione di metriche riproducibili, il flusso di elaborazione esplorato inizialmente in ambiente grafico è stato codificato in un'infrastruttura di script automatizzata. Questa suite di strumenti pilota i kernel di *Vivado HLS* e *Vivado* operando in modalità *batch*, eliminando completamente l'overhead d'interazione manuale.

L'infrastruttura di automazione sviluppata non si limita alla semplice sequenzialità di singoli comandi isolati, ma definisce un framework architetturale strutturato e orchestrato da un'unica interfaccia a riga di comando. Il flusso globale della **Design Space Exploration** è stato

strutturato e suddiviso in due rami funzionali principali, ciascuno mirato all'estrazione e alla validazione di specifiche metriche microarchitetturali e fisiche:

1. **Power Analysis Workflow (Ramo Principale):** Rappresenta il flusso di valutazione globale e approfondito dell'IP core. Il processo avvia la verifica funzionale tramite `csim`, procede con la sintesi in HLS e lancia la **C/RTL Co-Simulation** per convalidare il comportamento temporale a livello di pin, esportando contestualmente il pacchetto logico finale dell'IP. Il controllo passa poi in downstream all'ambiente **Vivado**, dove viene eseguita una sintesi logica del modulo in modalità **Out-Of-Context** (OOC)<sup>1</sup>. Poiché non è metodologicamente possibile avviare una simulazione post-sintesi direttamente sulla sintesi OOC pura del singolo blocco, l'IP viene istanziato all'interno di un apposito **wrapper** top-level per eseguire la simulazione post-sintesi per catturare i tassi di commutazione reali dei nodi interni, esportando il file di attività logica `.saif`. Questo tracciato viene infine re-iniettato nel motore di stima sulla sintesi OOC originaria<sup>2</sup>, consentendo di generare un report di potenza dinamica e statica estremamente accurato e isolato per il solo filtro FIR.
2. **Clock Analysis Workflow (Ramo Secondario):** Configurato come un flusso snello ad hoc per lo studio delle risposte temporali della logica combinatoria. Questo ramo si concentra esclusivamente sull'analisi di come la variazione del **vincolo del periodo di clock target** influenzi lo scheduling delle operazioni effettuato dal compilatore HLS (favorendo o limitando l'**Operation Chaining**). Operando esclusivamente su soluzioni e varianti algoritmiche già precedentemente validate dal punto di vista funzionale, lo script riesegue in modo mirato la sola fase di sintesi HLS applicando i *timing target* modificati, permettendo di mappare rapidamente la sensibilità dei percorsi critici e dei registri senza l'overhead computazionale delle simulazioni downstream.

Questi due workflow indipendenti sono stati inglobati in un singolo script generale di controllo, interamente richiamabile da riga di comando per centralizzare l'interfaccia utente. A livello tecnologico, l'automazione lato sistema operativo è affidata a file batch di Windows (`.bat`), mentre l'intera gestione interna dei tool Xilinx e delle simulazioni fisiche downstream è interamente codificata tramite script TCL.

Per ottimizzare la gestione dei dati, l'infrastruttura introduce uno script dedicato al **parsing** e all'aggregazione automatica dei report identificati come "rilevanti" per ogni specifica soluzione, centralizzandoli all'interno della directory "**reports**". Questo raggruppamento strutturato semplifica drasticamente sia i processi di **ispezione manuale** sia l'ingestione automatizzata da parte degli script *Python* di **visualizzazione dati**. Infine, un ulteriore script di controllo batch permette l'esecuzione massiva e sequenziale di più sintesi HLS consecutive, iterando automaticamente su uno *spettro di target di clock* differenti per mappare in maniera esaustiva i limiti di **slack** di ogni configurazione hardware sotto analisi.

---

<sup>1</sup>La sintesi **Out-Of-Context** (OOC) previene l'inferenza automatica dei buffer di **I/O** fisici (IBUF/OBUF) sui terminali del modulo. Questa modalità è essenziale per la caratterizzazione isolata di un *IP core*, poiché quest'ultimo è destinato a essere interconnesso internamente con altre logiche del sistema e non cablato direttamente ai pin esterni dell'**FPGA**.

<sup>2</sup>Sebbene l'inserimento del modulo in un *wrapper* di test possa generare lievi discrepanze gerarchiche rispetto alla logica isolata, ai fini dell'analisi energetica è sufficiente che il tasso di annotazione (il *matching* tra i nodi della simulazione `.saif` e la *netlist* sintetizzata) superi la soglia dell'**85%** affinché **Vivado** restituisca un report di potenza con livello di confidenza alto (*High Confidence*).

```

:: =====
::  USAGE
::
::  workflow.bat <PROJECT_NAME> <COMP_VERSION> <COMP_NAME>
::                [/wf <power|clk>]
::                [workflow-specific options...]
::
::  Positional (required):
::  PROJECT_NAME   Project directory name          (e.g., project01_FIR)
::  COMP_VERSION   HLS component directory name     (e.g., fir_baseline)
::  COMP_NAME      Top-level HLS function name      (e.g., fir)
::
::  Named (optional):
::  /wf           Workflow to run (default: power)
::                power = Full HLS pipeline + Vivado power analysis
::                clk   = Clock sweep: csim + synthesis + cosim
::
::  All remaining options are forwarded as-is to the selected workflow.
::  Run the individual scripts with no arguments to see their full usage.
::
::  Examples:
::  workflow.bat project01_FIR fir_baseline fir
::  workflow.bat project01_FIR fir_baseline fir /wf power /from 3
::  workflow.bat project01_FIR fir_baseline fir /wf clk 10ns
::  workflow.bat project01_FIR fir_baseline fir /wf clk 100MHz /from 2
:: =====

```

Codice 1: Struttura del comando per richiamare il workflow

### 2.3. Approccio all'Esplorazione «Greedy + Backtracking»

L'esplorazione dello spazio di design (*Design Space Exploration*) è stata governata da una metodologia combinata basata su strategie *Greedy* e meccanismi di *Backtracking*. Per massimizzare l'efficienza della ricerca, l'ordine di implementazione delle tecniche è stato strutturato gerarchicamente: si è partiti dalle modifiche più invasive a livello algoritmico (ristrutturazione del codice e *Loop Fission*), passando per la ridefinizione del *datapath* e la compressione dei registri (tramite i tipi esatti `ap_int` e le primitive *Shift Register*), fino ad arrivare al *refactoring* della memoria (*Array Partitioning*) e all'iniezione di parallelismo a grana fine spaziale e temporale (*Loop Unrolling* e *Loop Pipelining*).

Procedendo *step-by-step*, ogni singola ottimizzazione è stata valutata in isolamento. Se l'applicazione di una direttiva mostrava un miglioramento netto del bilancio globale (come l'*Area-Delay Product*) o permetteva di rispettare specifici vincoli architetturali, la soluzione veniva consolidata proseguendo al ramo successivo. In caso contrario, il meccanismo di *backtracking* prevedeva di scartare la direttiva testata, «tornare indietro» allo stato stabile precedente o tornare su un'altro ramo non esplorato e provare un'implementazione alternativa.

Tuttavia, per evitare di incorrere in minimi locali e scartare a priori configurazioni globalmente ottimali, l'esplorazione ha deliberatamente **mantenuto in vita alcune varianti «sub-ottimali»**. Alcune architetture, pur manifestando localmente un apparente degrado metrico (es. aumento ingiustificato dei *Flip-Flop* o mancato abbattimento della latenza), sono state portate avanti per innescare **sinergie latenti** con le ottimizzazioni successive. Ad esempio, l'introduzione diretta di direttive di parallelismo (*Unroll* e *Pipeline*) ha prodotto inizialmente risultati deludenti, pesantemente limitati dai colli di bottiglia legati ai rigidi vincoli di accesso simultaneo della *BRAM*. Questa evidenza ha motivato e reso necessario l'impiego

dell'ARRAY\_PARTITION: pur risultando debolmente incisivo o addirittura dispendioso in termini di area se valutato in isolamento, esso si è confermato il requisito propedeutico fondamentale per sbloccare le interfacce di memoria e permettere alle successive logiche di parallelismo di far esplodere le prestazioni temporali del filtro. Nel complesso, le opzioni esplorate coprono in maniera pressoché esaustiva i nodi decisionali chiave, scremando a monte solamente le varianti ridondanti o a priori identificate come non particolarmente rilevanti.

### 3. Metriche di Valutazione e Trade-off

#### 3.1. Metriche Prestazionali (Performance)

I dati prestazionali vengono estratti principalmente dal **Synthesis Report** generato da Vivado HLS (csynth.rpt e fir\_csynth.rpt). Le metriche fondamentali includono:

- **Latenza Iniziale ( $D_{\text{start}}$ ) e Totale ( $D_{\text{tot}}$ ):** misurata in colpi di clock, rappresenta il tempo richiesto per produrre l'output.
- **Initiation Interval (II):** indica il numero di cicli di clock prima che il sistema possa accettare nuovi dati di input.
- **Frequenza Massima Operativa ( $F_{\text{max}}$ ):** derivata dal periodo di clock stimato dal tool di sintesi.

Per le valutazioni di trade-off, la latenza in cicli viene convertita nel *Tempo di Esecuzione Totale* (Delay) tramite la formula:

$$T_{\text{tot}} = D_{\text{tot}} \times t_{\text{clk}}$$

#### 3.2. Metriche Hardware (Area)

L'occupazione delle risorse hardware viene confermata dai report di sintesi e di utilizzo post-sintesi logica (synth\_ooc/fir\_0\_utilization\_synth.rpt). Le metriche si basano sul conteggio esatto delle seguenti risorse fisiche sull'FPGA:

- **LUT** (Look-Up Tables) e **Flip-Flops** (FF) per la logica sequenziale e combinatoria.
- **BRAM** (Block RAM) per la memorizzazione dei dati e vettori.
- **DSP48** per le operazioni aritmetiche complesse (come le moltiplicazioni-accumulazioni).

Per confrontare in modo unificato il costo spaziale delle diverse architetture, si definisce un *Costo Totale dell'Area* pesato<sup>3</sup>:

$$A_{\text{tot}} = (w_{\text{LUT}} \times N_{\text{LUT}}) + (w_{\text{FF}} \times N_{\text{FF}}) + (w_{\text{BRAM}} \times N_{\text{BRAM}}) + (w_{\text{DSP}} \times N_{\text{DSP}})$$

#### 3.3. Metriche di Consumo (Power)

L'analisi del consumo energetico si basa sui **Power Report** (fir\_post-synth\_power\_report.txt e la sua variante .xml per l'estrazione dei dati a più *alta precisione*). Il consumo di potenza si divide in due componenti principali:

- **Potenza Statica:** dovuta alle correnti di dispersione ("leakage") intrinseche del dispositivo.
- **Potenza Dinamica:** associata all'attività del clock, alla commutazione dei segnali logici, all'uso delle memorie BRAM e dell'I/O.

Per ottenere stime accurate della potenza dinamica è di vitale importanza l'uso dei file *SAIF* (Switching Activity Interchange Format)<sup>4</sup>. Combinando la potenza totale stimata con il tempo di

<sup>3</sup>I pesi  $w$  vengono assegnati sulla base della scarsità relativa delle risorse fisiche presenti sull'FPGA target, definendoli come inversamente proporzionali alla loro disponibilità complessiva ( $\frac{1}{N_{\text{ris}}}$ ).

<sup>4</sup>Come spiegato in "Power Analysis Workflow (Ramo Principale)" 2.2

esecuzione per singola operazione e quello totale, si calcola l'*Energia per Operazione*<sup>5</sup> e l'*Energia Totale*:

$$E_{\text{op}} = P_{\text{tot}} \times T_{\text{op}}$$

$$E_{\text{tot}} = P_{\text{tot}} \times T_{\text{tot}}$$

### 3.4. Analisi dei Trade-off

Per comprendere l'efficienza reale delle ottimizzazioni, si ricorre al calcolo di specifici prodotti di valutazione:

- **Area-Delay Product (ADP):**

$$\text{ADP} = A_{\text{tot}} \times T_{\text{tot}}$$

Questa metrica rivela se l'applicazione di strategie esigenti in **termini di area** (come ad esempio un *Loop Unrolling* completo) abbia effettivamente prodotto un taglio dei tempi di esecuzione tale da giustificarne l'alto costo in hardware.

- **Energy-Delay Product (EDP):**

$$\text{EDP} = E_{\text{tot}} \times T_{\text{tot}}$$

L'EDP è considerata la metrica principe nel contesto del "*low-power design*". Essa penalizza severamente tutte quelle soluzioni architetturali che assorbono **enormi quantità di energia** per conseguire incrementi prestazionali solo marginali, permettendo di scartare iterazioni inefficienti.

## 4. Esplorazione dello Spazio di Design

### 4.1. Analisi dell'Architettura Baseline

#### 4.1.1. Implementazione Logica di Base

Il codice sorgente dell'IP (`fir.cpp` e `fir.hpp`) implementa l'algoritmo di filtraggio utilizzando un approccio procedurale classico in C/C++. I dati (campioni in ingresso, *buffer line* della storia dei campioni e i coefficienti `h[i]`) sono modellati tramite tipi nativi a larghezza standard (`int` a 32 bit). La logica interna si articola in un singolo ciclo `for`

```
...

fir_loop: for (int i = N-1; i >= 0; i--) {
    /* ... Operazioni sequenziali ... */
}

...
```

Codice 2: Ciclo `for` principale.

dove vengono eseguite due operazioni in modo *sequenziale*:

- **Shift Register Sequenziale:** Fa **scorrere** gli elementi dell'array di memoria della *buffer line* per far spazio all'ingresso corrente.

---

<sup>5</sup>Per "*singola operazione*" si intende il filtraggio di *un nuovo ingresso*.



```

...

// --- Shift REG ---
if (i == 0) {
    shift_reg[0] = x; // Insert the new data sample at the beginning
} else {
    shift_reg[i] = shift_reg[i - 1]; // Shift i-1 element to the right (drop the last
    element)
}

...

```

Codice 3: **Shift sequenziale** degli elementi nella *buffer line*.

- **Multiply-Accumulate (MAC)**: L'accumulo dei prodotti parziali calcolato iterativamente per tutti gli  $N$  *tap* del filtro.

```

...

// --- MAC ---
acc += shift_reg[i] * c[i]; // Perform MAC on the i-th element

...

```

Codice 4: **MAC** delle somme parziali.

Questa stesura, per quanto semanticamente ed algoritmicamente funzionale su architetture CPU, obbliga il sintetizzatore HLS a inferire un hardware guidato da una **FSM** (*Finite State Machine*) altamente sequenziale, in cui i cicli vincolano la schedulazione logica e dilatano enormemente i tempi di calcolo per ogni ciclo d'esecuzione.

#### 4.1.2. Testbench C/C++ Integrata

La robustezza formale dell'intera fase di DSE è garantita dall'implementazione del testbench `fir_tb.cpp`. Questo script non si limita a stimolare l'hardware, ma incorpora un **golden model** (un simulatore matematico di riferimento). Iniettando specifici vettori di test (es. risposte all'impulso, scalini o segnali sintetici casuali) all'interno del blocco in fase di test, il *testbench* ne calcola parallelamente l'uscita teorica e la confronta con l'effettivo output prodotto dalla funzione HLS. Se il modulo mantiene l'equivalenza matematica, la routine di test restituisce uno stato di successo (`return 0`). Questo framework *self-checking* viene conservato inalterato in tutte le varianti architetture del progetto, fungendo da rete di sicurezza fondamentale durante le co-simulazioni (*C/RTL Co-simulation*) per assicurare che ottimizzazioni hardware aggressive (come srotolamento o manipolazione dei tipi di dato) non introducano deviazioni logiche o troncamenti critici fatali.

#### 4.1.3. Disabilitazione delle Ottimizzazioni Automatiche

Per fini metodologici, in modo da valutare oggettivamente il peso delle singole tecniche di ottimizzazione, il compilatore HLS è stato depotenziato intenzionalmente tramite la configurazione `hls_config.cfg`. Nello specifico, l'alterazione e la disabilitazione di flag di controllo come `syn.compile.pipeline_loops` e `syn.array_partition.throughput_driven` impediscono alle euristiche dei tool Xilinx di ultima generazione (*Vitis HLS*) di applicare **in autonomia** direttive avanzate di parallelismo. Di conseguenza, la **baseline** incarna una conversione *C-to-RTL* rigorosamente 1:1. Questa disabilitazione impone il *backtracking* procedurale, scaricando sul progettista l'onere di implementare e orchestrare testualmente, in modo manuale e progressivo, l'iniezione dei **pragma** esplorati per soluzioni ottimizzate.

#### 4.1.4. Analisi del Testbench VHDL e dei Constraint di Sistema

Terminato lo *stage* esplorativo in ambito alto livello, l'IP generato approda in Vivado sfruttando l'architettura dei file presenti in `srcs\vivado\`:

- Il **Testbench VHDL** (`project01_FIR_00_baseline_tb.vhd`): Agisce come *wrapper* infrastrutturale per l'istanziamento fisico del modulo sintetizzato. Ha la funzione di emulare il master di sistema (un processore standard o una macchina a stati complessa), stimolando direttamente la rete dei segnali fisici di clock/reset e iniettando correttamente il protocollo handshaking estratto in fase HLS (tramite i *port signals* standard `ap_start`, `ap_done`, `ap_ready` e `ap_idle`).
- Il file **Constraint XDC** (`project01_FIR_00_baseline_clk.xdc`): Tramite i comandi fisici standard come `create_clock`, fissa il target temporale target (10.0ns o inferiore). Questo *constraint* istruisce il motore di posizionamento dell'FPGA (*Place & Route* per la sintesi Out-Of-Context) sui margini fisici dello *slack* e fissa il limite stringente su cui verranno estratti i metadati nel report di tolleranza termica dinamica (simulazione tracciata nei file `.saif`).

#### 4.1.5. Analisi delle Metriche di Valutazione e Limitazioni

##### Analisi preliminare

L'implementazione **baseline** del filtro FIR è caratterizzata da un'architettura puramente sequenziale e non ottimizzata. Il codice fa affidamento su un unico ciclo `for` principale (`fir_loop`) che esegue congiuntamente, iterazione dopo iterazione, sia la movimentazione dei dati all'interno dello shift register statico, sia la vera e propria operazione aritmetica di **Multiply-Accumulate** (MAC). L'assenza di direttive HLS (`pragma`) costringe il sintetizzatore a implementare il comportamento C/C++ di default: viene istanziata una singola unità di calcolo che viene riutilizzata in *Time-Multiplexing* per tutte le N iterazioni (11 *tap* del filtro), mentre i dati elaborati mantengono l'ampiezza standard a 32 *bit* imposta dai tipi generici `int`.

#### Timing and Latency

| Metric                       | At target clock                           | At estimated clock                        |
|------------------------------|-------------------------------------------|-------------------------------------------|
| Target clock period          | 10.000 ns                                 | —                                         |
| Estimated clock / $F_{\max}$ | —                                         | 6.210 ns                                  |
| Max achievable $F_{\max}$    | —                                         | <b>161.030 MHz</b>                        |
| Clock uncertainty            | 2.7 ns                                    |                                           |
| Latency min                  | 67cyc → 670.0 ns<br>(0.6700 $\mu$ s)      | 67cyc → 416.1 ns<br>(0.4161 $\mu$ s)      |
| Latency max                  | 67cyc → 670.0 ns<br>(0.6700 $\mu$ s)      | 67cyc → 416.1 ns<br>(0.4161 $\mu$ s)      |
| CoSim latency avg            | 67cyc → 670.0 ns<br>(0.6700 $\mu$ s)      | 67cyc → 416.1 ns<br>(0.4161 $\mu$ s)      |
| CoSim total execution        | 2039cyc → 20390.0 ns<br>(20.3900 $\mu$ s) | 2039cyc → 12662.2 ns<br>(12.6622 $\mu$ s) |

## Resource Utilisation

| Resource                                                    | Used | Available | Utilisation     |
|-------------------------------------------------------------|------|-----------|-----------------|
| LUT                                                         | 192  | 53200     | 0.36 %          |
| FF                                                          | 198  | 106400    | 0.19 %          |
| BRAM                                                        | 0    | 280       | 0.00 %          |
| DSP                                                         | 2    | 220       | 0.91 %          |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |      |           | <b>0.014561</b> |

## Power and Energy

| Metric                                                                                    | Value                    |
|-------------------------------------------------------------------------------------------|--------------------------|
| Static power $P_{\text{static}}$                                                          | 0.103 W                  |
| Dynamic power $P_{\text{dynamic}}$                                                        | 0.001 W                  |
| <b>Total power <math>P_{\text{total}}</math></b>                                          | <b>0.104 W</b>           |
| Energy/op $E_{\text{op}} = P_{\text{total}} \times \text{Latency}_{\text{max,est}}$       | $4.327 \times 10^{-8}$ J |
| Total energy $E_{\text{total}} = P_{\text{total}} \times \text{Latency}_{\text{tot,est}}$ | $1.317 \times 10^{-6}$ J |

## Trade-off Merits

| Metric                     | Value                   |
|----------------------------|-------------------------|
| Area-Delay Product (ADP)   | 184.3741                |
| Energy-Delay Product (EDP) | $1.6674 \times 10^{-2}$ |

### Analisi delle metriche

I dati rispettano fedelmente le aspettative per una configurazione iniziale di questo tipo. L'area hardware occupata è estremamente ridotta: vengono impiegati solamente 2 blocchi DSP, 192 LUT e 198 FF, confermando l'elevato livello di riutilizzo delle risorse. Tuttavia, questo drastico risparmio di area si paga con prestazioni temporali scadenti. La latenza per l'elaborazione di un singolo campione si attesta a *67 colpi di clock* (in media *6 cicli* per ognuno degli *11 tap*), portando l'Initiation Interval (II) a 68. Questo significa che il throughput è gravemente limitato, poiché il filtro non può accettare un nuovo input fino alla completa conclusione dell'elaborazione precedente. Il periodo di clock stimato (6.21 ns) rispetta comodamente il constraint di 10.0 ns, mantenendo la potenza dinamica molto bassa (0.001 W), ma il lungo tempo di esecuzione complessivo penalizza comunque le metriche di trade-off come l'**Energy-Delay Product** (EDP).

### Conclusioni

La criticità principale di questa architettura risiede nel collo di bottiglia strutturale causato dal datapath sequenziale, unito a uno spreco energetico e logico derivante dall'uso di tipi di dato sovradimensionati (*32 bit* per coefficienti che variano tra  $-91$  e  $500$ ). Il primo passo consisterà nell'applicare il **Loop Fission** per separare il ciclo di shift da quello di accumulo, permettendo al sintetizzatore di ottimizzarli indipendentemente. Subito dopo, sarà cruciale introdurre la **Bitwidth Optimization** tramite la libreria `ap_int` per restringere i registri al numero di bit strettamente necessario. Infine, l'introduzione progressiva del **Loop Unrolling** (parallelismo spaziale) e del **Loop Pipelining** (parallelismo temporale) punterà ad abbattere drasticamente la latenza e a portare l'Initiation Interval al target ideale di 1 *colpo di clock*.

## 4.2. Ottimizzazioni Architettureali

### 4.2.1. Eliminazione dei branching: «Code Hoisting»

Il primo passo per l'ottimizzazione del codice sorgente rispetto all'implementazione **baseline** consiste nella rimozione delle inefficienze strutturali a livello di datapath di controllo. Nel ciclo originario, la presenza del costrutto condizionale `if (i == 0)` costringeva il sintetizzatore HLS a inferire logica di controllo aggiuntiva (come multiplexer e comparatori) per valutare il salto ad ogni singola iterazione, nonostante la condizione si verificasse, di fatto, per un unico e ben noto valore dell'indice.

Attraverso la tecnica del **Code Hoisting**, la logica legata all'inserimento del nuovo campione d'ingresso è stata fisicamente estratta e portata al di fuori del corpo del ciclo principale. Il **loop** è stato di conseguenza modificato nei suoi limiti (con condizione di terminazione `i > 0`):

```
...

// Sequential shift and MAC (Multiply-Accumulate) operations
fir_loop: for (int i = N-1; i > 0; i--) {
    // --- Shift REG ---
    shift_reg[i] = shift_reg[i - 1]; // Shift i-1 element to the right (drop the last
    element)

    // --- MAC ---
    acc += shift_reg[i] * c[i]; // Perform MAC on the i-th element
}
```

...

Codice 5: Ciclo `for` ottimizzato e liberato dai vincoli condizionali interni.

In questo modo, il ciclo elabora in maniera omogenea e ininterrotta solamente lo *shift* e il calcolo del *MAC* per i dati storici già presenti nella *buffer line*. Subito dopo l'uscita dal ciclo, viene gestito esplicitamente il caso limite per l'indice  $i = 0$ :

```

...

/** OPTIMIZATION(code-hoisting): removed conditional check in the loop
// --- Shift REG ---
shift_reg[0] = x; // Insert the new data sample at the beginning

// --- MAC ---
acc += shift_reg[0] * c[0]; // Perform MAC on the first element

...

```

Codice 6: Operazioni di *shift* e *accumulo* per  $i = 0$  eseguite sequenzialmente fuori dal ciclo.

Questa pre-elaborazione a livello C/C++ ha un impatto diretto sull'hardware generato: allevia la **FSM** dal controllo del *branch* ripetitivo, minimizza lo spreco di risorse logiche e crea un datapath aritmetico molto più regolare, un requisito essenziale per massimizzare i benefici delle direttive di **Pipelining** e **Unrolling** che verranno introdotte nelle fasi successive del progetto.

#### 4.2.1.1. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione **01\_code-hoisting** introduce una prima ottimizzazione architetturale mirata a snellire la logica di controllo inferita dal sintetizzatore. L'applicazione della tecnica del **Code Hoisting** ha lo scopo di rimuovere il costoso overhead hardware causato dalla continua valutazione di condizioni limite durante il normale flusso iterativo. Da questa operazione ci aspettiamo la generazione di una *FSM* notevolmente più lineare e ininterrotta, liberata dalla complessa logica di branching (come *multiplexer* e *comparatori* di controllo) che appesantiva la baseline. L'obiettivo primario è ottenere un datapath più regolare che ottimizzi la schedulazione logica e riduca i *colpi di clock* sprecati, creando al contempo le condizioni architetturali ideali per massimizzare l'efficacia delle future direttive di **parallelizzazione**.

##### Timing

| Metric                       | 00 — Baseline      | 01 — Code hoisting | $\Delta$ |
|------------------------------|--------------------|--------------------|----------|
| Target clock                 | 10.000 ns          | 10.000 ns          | —        |
| Estimated clock / $F_{\max}$ | 6.210 ns           | 6.424 ns           | ▲ 3.45 % |
| Max achievable $F_{\max}$    | <b>161.030 MHz</b> | <b>155.670 MHz</b> | ▼ 3.33 % |
| Clock uncertainty            | 2.7 ns             | 2.7 ns             | —        |

##### Latency and Throughput

| Metric                        | 00 — Baseline                        | 01 — Code hoisting                   | $\Delta$  |
|-------------------------------|--------------------------------------|--------------------------------------|-----------|
| Latency max @ target clock    | 67cyc → 670.0 ns<br>(0.6700 $\mu$ s) | 41cyc → 410.0 ns<br>(0.4100 $\mu$ s) | ▼ 38.81 % |
| Latency max @ estimated clock | 67cyc → 416.1 ns<br>(0.4161 $\mu$ s) | 41cyc → 263.4 ns<br>(0.2634 $\mu$ s) | ▼ 36.70 % |

|                                     |                                           |                                           |                  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|------------------|
| CoSim total exec. @ target clock    | 2039cyc → 20390.0 ns<br>(20.3900 $\mu$ s) | 1259cyc → 12590.0 ns<br>(12.5900 $\mu$ s) | ▼ <b>38.25 %</b> |
| CoSim total exec. @ estimated clock | 2039cyc → 12662.2 ns<br>(12.6622 $\mu$ s) | 1259cyc → 8087.8 ns<br>(8.0878 $\mu$ s)   | ▼ <b>36.13 %</b> |

## Resource Utilisation

| Metric                                                      | 00 — Baseline | 01 — Code hoisting | $\Delta$         |
|-------------------------------------------------------------|---------------|--------------------|------------------|
| LUT                                                         | 192 (0.36 %)  | 295 (0.55 %)       | ▲ <b>53.65 %</b> |
| FF                                                          | 198 (0.19 %)  | 193 (0.18 %)       | ▼ <b>2.53 %</b>  |
| BRAM                                                        | 0 (0.00 %)    | 0 (0.00 %)         | —                |
| DSP                                                         | 2 (0.91 %)    | 2 (0.91 %)         | = <b>0.00 %</b>  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |               |                    | ▲ <b>12.97 %</b> |

## Power and Energy

| Metric                                           | 00 — Baseline            | 01 — Code hoisting       | $\Delta$         |
|--------------------------------------------------|--------------------------|--------------------------|------------------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = <b>0.00 %</b>  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                  | = <b>0.00 %</b>  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.104 W</b>           | = <b>0.00 %</b>  |
| Energy/op $E_{\text{op}}$                        | $4.327 \times 10^{-8}$ J | $2.739 \times 10^{-8}$ J | ▼ <b>36.70 %</b> |
| Total energy $E_{\text{total}}$                  | $1.317 \times 10^{-6}$ J | $8.411 \times 10^{-7}$ J | ▼ <b>36.13 %</b> |

## Figures of Merit

| Metric                     | 00 — Baseline           | 01 — Code hoisting      | $\Delta$         |
|----------------------------|-------------------------|-------------------------|------------------|
| Area-Delay Product (ADP)   | 184.3741                | 133.0446                | ▼ <b>27.84 %</b> |
| Energy-Delay Product (EDP) | $1.6674 \times 10^{-2}$ | $6.8029 \times 10^{-3}$ | ▼ <b>59.20 %</b> |

### Analisi delle metriche

I dati dimostrano chiaramente l'efficacia di questa operazione sulle prestazioni temporali, superando le rigidità della **baseline**. La latenza per la singola elaborazione crolla da 67 a 41 **colpi di clock** (una riduzione di quasi il 39%), portando di conseguenza l'Initiation Interval (II) da 68 a 42. Il numero di *cicli totali* scende proporzionalmente da 2039 a 1259. Questo netto incremento del throughput richiede un moderato sacrificio in termini di area logica: le LUT impiegate salgono a 295 (un aumento del 53% rispetto alle 192 della versione

precedente), come fisiologica conseguenza della duplicazione logica del blocco per l'indice 0 estratto dal ciclo. Nonostante il periodo di clock stimato si innalzi leggermente a 6.424 ns, il forte abbattimento del tempo totale di esecuzione porta l'**Energy-Delay Product** (EDP) a migliorare drasticamente, scendendo del 59% (da circa 0.0166 a 0.0068).

## Conclusioni

Sebbene la rimozione dell'overhead di controllo abbia snellito la *FSM* ed esaltato le prestazioni temporali rispetto all'implementazione puramente C/C++ di default, l'architettura soffre ancora di limitazioni intrinseche. Lo *shift* dei registri statici e le operazioni di **MAC** restano accoppiati nello stesso blocco iterativo, frenando il sintetizzatore HLS dall'ottimizzarli in parallelo. Il prossimo passo dunque è l'implementazione del **Loop Fission**, scindendo il processo in due cicli distinti. Fatto ciò, sarà imprescindibile avviare la **Bitwidth Optimization** tramite i tipi arbitrari `ap_int` per contenere l'uso delle risorse in vista dell'applicazione del **Loop Unrolling** e del **Loop Pipelining**.

### 4.2.2. Separazione dei flussi: «Loop Fission»

Il passo successivo nel processo di ottimizzazione consiste nell'isolare le operazioni logiche all'interno dell'algoritmo. Nel codice originario e nella prima revisione, lo *shift* dei dati e l'operazione di *Multiply-Accumulate* (MAC) erano accoppiati nello stesso blocco iterativo.

Attraverso la tecnica del **Loop Fission**, il ciclo è stato scisso in due *loop* distinti e specializzati. Il primo ciclo (`shift_loop`) si occupa esclusivamente dell'aggiornamento della *buffer line*, mentre il secondo ciclo (`mac_loop`) gestisce puramente l'accumulo aritmetico:

```
...

/* OPTIMIZATION(loop-fission): loop separation for the shift and MAC operations
// --- Shift REG ---
shift_loop: for (int i = N-1; i > 0; i--) {
    shift_reg[i] = shift_reg[i - 1]; // Shift i-1 element to the right (drop the
last element)
}
/* OPTIMIZATION(code-hoisting): removed conditional check in the loop
shift_reg[0] = x; // Insert the new data sample at the beginning

// --- MAC ---
mac_loop: for (int i = 0; i < N; i++) {
    acc += shift_reg[i] * c[i]; // Perform MAC on the i-th element
}

...
```

...

Codice 7: Codice sorgente dopo l'applicazione del **Loop Fission**, con cicli separati per *shift* e *MAC*.

Questa trasformazione, pur non fornendo un beneficio prestazionale immediato (poiché di default i cicli vengono ancora eseguiti in sequenza dal sintetizzatore), funge da pre-requisito architetturale critico. Disaccoppiando le dipendenze di *read/write* sui registri interni, il sintetizzatore HLS è ora in grado di valutare e ottimizzare i due *datapath* in maniera completamente indipendente nelle fasi successive.

#### 4.2.2.1. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione `02_loop-fission` mira a modularizzare l'architettura. L'aspettativa a livello hardware non è un incremento delle prestazioni temporali: la separazione del flusso in due cicli impone infatti la creazione di una *FSM* che deve iterare e transitare tra due blocchi logici eseguiti in stretta serie. Ci aspettiamo quindi un fisiologico peggioramento della latenza (dovuto all'overhead sequenziale di cicli multipli) compensato però da una semplificazione della logica di controllo interna a ciascun *loop*. L'obiettivo primario è porre le basi strutturali per massimizzare l'efficacia delle imminenti direttive di parallelizzazione.

##### Timing

| Metric                       | 01 — Code hoisting | 02 — Loop fission  | $\Delta$ |
|------------------------------|--------------------|--------------------|----------|
| Target clock                 | 10.000 ns          | 10.000 ns          | —        |
| Estimated clock / $F_{\max}$ | 6.424 ns           | 6.210 ns           | ▼ 3.33 % |
| Max achievable $F_{\max}$    | <b>155.670 MHz</b> | <b>161.030 MHz</b> | ▲ 3.44 % |
| Clock uncertainty            | 2.7 ns             | 2.7 ns             | —        |

##### Latency and Throughput

| Metric                              | 01 — Code hoisting                        | 02 — Loop fission                         | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 41cyc → 410.0 ns<br>(0.4100 $\mu$ s)      | 66cyc → 660.0 ns<br>(0.6600 $\mu$ s)      | ▲ 60.98 % |
| Latency max @ estimated clock       | 41cyc → 263.4 ns<br>(0.2634 $\mu$ s)      | 66cyc → 409.9 ns<br>(0.4099 $\mu$ s)      | ▲ 55.61 % |
| CoSim total exec. @ target clock    | 1259cyc → 12590.0 ns<br>(12.5900 $\mu$ s) | 2009cyc → 20090.0 ns<br>(20.0900 $\mu$ s) | ▲ 59.57 % |
| CoSim total exec. @ estimated clock | 1259cyc → 8087.8 ns<br>(8.0878 $\mu$ s)   | 2009cyc → 12475.9 ns<br>(12.4759 $\mu$ s) | ▲ 54.26 % |

##### Resource Utilisation

| Metric                                                      | 01 — Code hoisting | 02 — Loop fission | $\Delta$  |
|-------------------------------------------------------------|--------------------|-------------------|-----------|
| LUT                                                         | 295 (0.55 %)       | 214 (0.40 %)      | ▼ 27.46 % |
| FF                                                          | 193 (0.18 %)       | 199 (0.19 %)      | ▲ 3.11 %  |
| BRAM                                                        | 0 (0.00 %)         | 0 (0.00 %)        | —         |
| DSP                                                         | 2 (0.91 %)         | 2 (0.91 %)        | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                    |                   | ▼ 8.91 %  |



## Power and Energy

| Metric                                           | 01 — Code hoisting       | 02 — Loop fission        | $\Delta$  |
|--------------------------------------------------|--------------------------|--------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                  | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.104 W</b>           | = 0.00 %  |
| Energy/op $E_{\text{op}}$                        | $2.739 \times 10^{-8}$ J | $4.263 \times 10^{-8}$ J | ▲ 55.61 % |
| Total energy $E_{\text{total}}$                  | $8.411 \times 10^{-7}$ J | $1.297 \times 10^{-6}$ J | ▲ 54.26 % |

## Figures of Merit

| Metric                     | 01 — Code hoisting      | 02 — Loop fission       | $\Delta$   |
|----------------------------|-------------------------|-------------------------|------------|
| Area-Delay Product (ADP)   | 133.0446                | 186.9387                | ▲ 40.51 %  |
| Energy-Delay Product (EDP) | $6.8029 \times 10^{-3}$ | $1.6187 \times 10^{-2}$ | ▲ 137.95 % |

### Analisi delle metriche

Come previsto teoricamente, rispetto alla soluzione precedente (**01\_code-hoisting**), le metriche temporali subiscono una flessione evidente. La latenza per l'elaborazione di un singolo campione risale da 41 a **66 colpi di clock** (un aumento del 61%), riportando l'Initiation Interval (II) a 67. Parallelamente, il numero di *cicli totali* di esecuzione sale da 1259 a 2009. A fronte di questo rallentamento, la separazione dei domini ha però permesso al sintetizzatore di inferire hardware molto meno complesso: le risorse logiche vedono un netto calo, con le LUT che passano da 295 a 214 (riduzione del  $\sim 27\%$ ). Se estendiamo il confronto alla **baseline** iniziale, notiamo come le prestazioni temporali siano quasi tornate al punto di partenza (latenza di 66 contro 67, cicli totali 2009 contro 2039). L'area logica risulta lievemente superiore (214 LUT contro le 192 della baseline) a causa della conservazione logica del **Code Hoisting**, ma il periodo di clock stimato rientra esattamente al valore originale di 6.21 ns, indicando un *Critical Path* rilassato.

### Conclusioni

Il **Loop Fission** si conferma come una *enabling transformation*: applicato singolarmente degrada il throughput temporale immediato a causa della schedulazione in serie dei cicli, ma ha il grande pregio di isolare i blocchi logici e ridurre l'area combinatoria (LUT). Tuttavia, per sfruttare appieno il potenziale di questa progressione senza l'overhead dei cicli sequenziali, il prossimo passo consisterà nell'applicazione della **Bitwidth Optimization** direttamente sulla versione **01\_code-hoisting**. Questa analisi comparativa parallela permetterà di verificare se sia possibile estrarre ulteriori prestazioni e un migliore bilanciamento energetico dalla soluzione precedente, stringendo i tipi nativi da 32 *bit* all'effettivo

intervallo dinamico tramite la libreria `ap_int`, prima di procedere con le trasformazioni di parallelismo.

## 4.3. Ottimizzazioni dei Tipi

### 4.3.1. Ottimizzazione del Datapath: «Bitwidth Optimization»

L'analisi delle soluzioni precedenti ha evidenziato come l'utilizzo dei tipi primitivi nativi (`int` a 32 *bit*) rappresenti una grave inefficienza energetica e d'area, poiché le ampiezze dei registri risultano ampiamente sovradimensionate rispetto alla reale dinamica dei segnali. Per superare questa criticità, è stata implementata la **Bitwidth Optimization** includendo la libreria ad precisione arbitraria `<ap_int.h>`. Questa tecnica è stata applicata direttamente sulla variante *01\_code-hoisting* per esplorare se la riduzione del datapath potesse sbloccare ulteriori margini di performance senza l'overhead di controllo introdotto dal **Loop Fission**. I coefficienti del filtro sono stati ridotti ad un'ampiezza di 10 *bit signed*, i campioni d'ingresso a 12 *bit unsigned*<sup>6</sup>, mentre l'accumulatore è stato dimensionato a 26 *bit* (calcolati a tempo di compilazione tramite espressioni `constexpr` per evitare fenomeni di *overflow*).

#### 4.3.1.1. Adeguamento dell'Ambiente di Validazione

L'introduzione dei tipi a precisione arbitraria ha richiesto un corrispondente refactoring del testbench di verifica (`fir_tb.cpp`). Per garantire la coerenza dei dati sia nella fase di **C-Simulation** sia nella successiva **C/RTL Co-Simulation** di Vivado, le variabili di stimolo e di cattura dei risultati sono state convertite dai tipi nativi `int` ai tipi personalizzati `in_data_t` e `out_data_t` definiti nell'header di progetto. Questa modifica ha ridefinito esclusivamente l'ampiezza dei bus di interfaccia, lasciando inalterata la logica algoritmica del **Golden Model** software. In questo modo è stato preservato l'esatto allineamento dei bit durante lo scambio dei vettori di test, azzerando il rischio di *truncamenti hardware* imprevisti o di falsi negativi nei report di validazione RTL.

#### 4.3.1.2. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione *03\_ap-int* si concentra sulla massimizzazione dell'efficienza della logica combinatoria e sequenziale. Restringendo i bus aritmetici e i registri interni all'intervallo dinamico strettamente necessario, ci aspettiamo un drastico crollo nell'occupazione di area hardware (LUT e Flip-Flop) e un netto rilassamento dei ritardi di propagazione lungo le interconnessioni. L'obiettivo primario di questa sessione è dimostrare come la compressione del datapath permetta di abbattere il *periodo di clock stimato*, riducendo drasticamente il *Critical Path* dell'IP core e ponendo le basi per un consumo energetico ottimizzato.

## Timing

| Metric       | 02 — Loop fission | 03 — Bitwidth opt<br>(on v01) | $\Delta$ |
|--------------|-------------------|-------------------------------|----------|
| Target clock | 10.000 ns         | 10.000 ns                     | —        |

<sup>6</sup>I dati da filtrare sono stati considerati come se provenienti da un **ADC**, tipicamente a 12 *bit*.

|                              |                    |                    |           |
|------------------------------|--------------------|--------------------|-----------|
| Estimated clock / $F_{\max}$ | 6.210 ns           | 3.132 ns           | ▼ 49.57 % |
| Max achievable $F_{\max}$    | <b>161.030 MHz</b> | <b>319.280 MHz</b> | ▲ 98.27 % |
| Clock uncertainty            | 2.7 ns             | 2.7 ns             | —         |

### Latency and Throughput

| Metric                              | 02 — Loop fission                         | 03 — Bitwidth opt<br>(on v01)             | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 66cyc → 660.0 ns<br>(0.6600 $\mu$ s)      | 54cyc → 540.0 ns<br>(0.5400 $\mu$ s)      | ▼ 18.18 % |
| Latency max @ estimated clock       | 66cyc → 409.9 ns<br>(0.4099 $\mu$ s)      | 54cyc → 169.1 ns<br>(0.1691 $\mu$ s)      | ▼ 58.74 % |
| CoSim total exec. @ target clock    | 2009cyc → 20090.0 ns<br>(20.0900 $\mu$ s) | 1649cyc → 16490.0 ns<br>(16.4900 $\mu$ s) | ▼ 17.92 % |
| CoSim total exec. @ estimated clock | 2009cyc → 12475.9 ns<br>(12.4759 $\mu$ s) | 1649cyc → 5164.7 ns<br>(5.1647 $\mu$ s)   | ▼ 58.60 % |

### Resource Utilisation

| Metric                                                      | 02 — Loop fission | 03 — Bitwidth opt<br>(on v01) | $\Delta$  |
|-------------------------------------------------------------|-------------------|-------------------------------|-----------|
| LUT                                                         | 214 (0.40 %)      | 120 (0.23 %)                  | ▼ 43.93 % |
| FF                                                          | 199 (0.19 %)      | 77 (0.07 %)                   | ▼ 61.31 % |
| BRAM                                                        | 0 (0.00 %)        | 0 (0.00 %)                    | —         |
| DSP                                                         | 2 (0.91 %)        | 2 (0.91 %)                    | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                   |                               | ▼ 19.45 % |

### Power and Energy

| Metric                                           | 02 — Loop fission        | 03 — Bitwidth opt<br>(on v01) | $\Delta$  |
|--------------------------------------------------|--------------------------|-------------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                       | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                       | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.103 W</b>                | ▼ 0.96 %  |
| Energy/op $E_{\text{op}}$                        | $4.263 \times 10^{-8}$ J | $1.742 \times 10^{-8}$ J      | ▼ 59.13 % |
| Total energy $E_{\text{total}}$                  | $1.297 \times 10^{-6}$ J | $5.320 \times 10^{-7}$ J      | ▼ 59.00 % |

### Figures of Merit

| Metric                |         | 02 — Loop fission       | 03 — Bitwidth opt<br>(on v01) | $\Delta$  |
|-----------------------|---------|-------------------------|-------------------------------|-----------|
| Area-Delay<br>(ADP)   | Product | 186.9387                | 62.3375                       | ▼ 66.65 % |
| Energy-Delay<br>(EDP) | Product | $1.6187 \times 10^{-2}$ | $2.7474 \times 10^{-3}$       | ▼ 83.03 % |

#### Analisi delle metriche

Il confronto con la soluzione precedente (`02_loop-fission`) conferma le aspettative teoriche. Sotto il profilo delle risorse fisiche, si assiste ad un abbattimento straordinario dell'area: le LUT crollano da 214 a sole 120 (un risparmio del 43.9%) e i Flip-Flop scendono da 199 a 77 (riduzione del 61.3%). Sul fronte temporale, la latenza si riduce da 66 a 54 *colpi di clock* (riduzione del 18.2%) con i cicli totali di co-simulazione che scendono a 1649. L'effetto più netto è evidenziato sul *periodo di clock stimato*, che dimezza passando da 6.21 ns a soli 3.132 ns (50% di velocità in più). Se confrontiamo questo dato con la variante `01_code-hoisting` (latenza 41 cicli a 6.424 ns), notiamo che sebbene l'ottimizzazione dei bit aumenti la latenza algoritmica a 54 cicli a causa di una schedulazione sequenziale più rigida imposta dal sintetizzatore, il tempo reale di esecuzione assoluto crolla da 263.38 ns a 169.12 ns, segnando un guadagno netto del 35.8% in termini di puro *Delay* operativo e surclassando ogni configurazione precedente.

#### Conclusioni

La scelta strategica di operare la **Bitwidth Optimization** direttamente sull'architettura compatta del **Code Hoisting** si è rivelata vincente: ha dimostrato che il ridimensionamento dei tipi nativi è in grado di estrarre prestazioni superiori rispetto al **Loop Fission**, riducendo contemporaneamente l'area al minimo. Il datapath è ora perfettamente ottimizzato e privo di ridondanze logiche. Tuttavia, la persistenza di un'architettura a shift register statico basata su array continua a vincolare l'Initiation Interval (II) a 55 cicli. Per analizzare se sia possibile aggirare questo collo di bottiglia strutturale, la prossima soluzione esplorerà l'applicazione della libreria nativa `ap_shift_reg`, valutando se l'inferenza di shift register dedicati possa ottenere ulteriori miglioramenti prima di procedere con le direttive di **parallelismo**.

#### 4.3.2. Ottimizzazione della Memoria di Stato: Libreria «`ap_shift_reg`»

L'esplorazione delle architetture sequenziali prosegue focalizzandosi sulla struttura di storage dei campioni storici. Nelle versioni precedenti, la linea di ritardo era affidata ad array statici standard, costringendo il sintetizzatore a implementare manualmente la logica di movimentazione. Per massimizzare l'efficienza, in questa soluzione viene analizzato l'utilizzo della libreria `<ap_shift_reg.h>`, che mette a disposizione la classe template `ap_shift_reg`. L'introduzione di questa struttura ha lo scopo di forzare l'inferenza di primitive hardware dedicate e pre-ottimizzate, verificando se l'abbandono della gestione nativa degli array possa ottimizzare ulteriormente il datapath ed estrarre margini prestazionali residui prima di sbloccare il **parallelismo**.

### 4.3.3. Valutazione dei miglioramenti

#### Analisi preliminare

La soluzione `04_ap-shift-reg` sposta l'asse d'ottimizzazione sui blocchi di memoria interni. L'aspettativa hardware risiede nell'inferenza mirata delle macro-primitive dell'FPGA note come *SRL* (*Shift Register LUT*), le quali permettono di implementare l'intera catena di ritardo sfruttando i registri interni alle celle logiche configurabili anziché banchi di registri discreti. Ci aspettiamo una contrazione importante delle risorse combinatorie globali e un forte alleggerimento del routing delle interconnessioni, con un conseguente incremento della frequenza massima operativa ( $F_{\max}$ ).

#### Timing

| Metric                       | 03 — Bitwidth opt<br>(on v01) | 04 — Library<br>ap_shift_reg | $\Delta$  |
|------------------------------|-------------------------------|------------------------------|-----------|
| Target clock                 | 10.000 ns                     | 10.000 ns                    | —         |
| Estimated clock / $F_{\max}$ | 3.132 ns                      | 2.686 ns                     | ▼ 14.24 % |
| Max achievable $F_{\max}$    | <b>319.280 MHz</b>            | <b>372.300 MHz</b>           | ▲ 16.61 % |
| Clock uncertainty            | 2.7 ns                        | 2.7 ns                       | —         |

#### Latency and Throughput

| Metric                              | 03 — Bitwidth opt<br>(on v01)             | 04 — Library<br>ap_shift_reg              | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 54cyc → 540.0 ns<br>(0.5400 $\mu$ s)      | 56cyc → 560.0 ns<br>(0.5600 $\mu$ s)      | ▲ 3.70 %  |
| Latency max @ estimated clock       | 54cyc → 169.1 ns<br>(0.1691 $\mu$ s)      | 56cyc → 150.4 ns<br>(0.1504 $\mu$ s)      | ▼ 11.06 % |
| CoSim total exec. @ target clock    | 1649cyc → 16490.0 ns<br>(16.4900 $\mu$ s) | 1709cyc → 17090.0 ns<br>(17.0900 $\mu$ s) | ▲ 3.64 %  |
| CoSim total exec. @ estimated clock | 1649cyc → 5164.7 ns<br>(5.1647 $\mu$ s)   | 1709cyc → 4590.4 ns<br>(4.5904 $\mu$ s)   | ▼ 11.12 % |

#### Resource Utilisation

| Metric | 03 — Bitwidth opt<br>(on v01) | 04 — Library<br>ap_shift_reg | $\Delta$  |
|--------|-------------------------------|------------------------------|-----------|
| LUT    | 120 (0.23 %)                  | 107 (0.20 %)                 | ▼ 10.83 % |
| FF     | 77 (0.07 %)                   | 114 (0.11 %)                 | ▲ 48.05 % |
| BRAM   | 0 (0.00 %)                    | 0 (0.00 %)                   | —         |
| DSP    | 2 (0.91 %)                    | 1 (0.45 %)                   | ▼ 50.00 % |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ | ▼ 36.80 % |
|-------------------------------------------------------------|-----------|

## Power and Energy

| Metric                                           | 03 — Bitwidth opt<br>(on v01) | 04 — Library<br>ap_shift_reg | $\Delta$  |
|--------------------------------------------------|-------------------------------|------------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                       | 0.103 W                      | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                       | 0.001 W                      | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>                | <b>0.103 W</b>               | = 0.00 %  |
| Energy/op $E_{\text{op}}$                        | $1.742 \times 10^{-8}$ J      | $1.549 \times 10^{-8}$ J     | ▼ 11.06 % |
| Total energy $E_{\text{total}}$                  | $5.320 \times 10^{-7}$ J      | $4.728 \times 10^{-7}$ J     | ▼ 11.12 % |

## Figures of Merit

| Metric                     | 03 — Bitwidth opt<br>(on v01) | 04 — Library<br>ap_shift_reg | $\Delta$  |
|----------------------------|-------------------------------|------------------------------|-----------|
| Area-Delay Product (ADP)   | 62.3375                       | 35.0154                      | ▼ 43.83 % |
| Energy-Delay Product (EDP) | $2.7474 \times 10^{-3}$       | $2.1704 \times 10^{-3}$      | ▼ 21.00 % |

### Analisi delle metriche

L'analisi comparativa rispetto alla soluzione precedente (03\_ap-int) evidenzia un trade-off di grande interesse architetturale. Dal punto di vista temporale, si registra un lieve incremento della latenza algoritmica, che passa da 54 a 56 **colpi di clock** (un incremento del 3.7%), con i cicli totali di co-simulazione che salgono a 1709. Tuttavia, la forte regolarizzazione del datapath operata dalla libreria abbatte sensibilmente il *Critical Path*: il periodo di clock stimato si riduce ulteriormente da 3.132 ns a ben 2.686 ns (un guadagno del 14.2%). Di conseguenza, il tempo assoluto di elaborazione reale crolla da 169.13 ns a 150.42 ns, registrando un miglioramento netto dell'efficienza temporale del 11.1%. Sotto il profilo dell'occupazione d'area, l'uso di strutture dedicate riduce le **LUT** da 120 a 107 (risparmio del 10.8%) e dimezza l'uso di blocchi aritmetici avanzati, vedendo i **DSP** scendere da 2 a una sola 1 istanza hardware. Si rileva d'altra parte un incremento fisiologico dei **Flip-Flop**, che passano da 77 a 114 a causa dell'allocazione dei registri di pipeline interni alla struttura di shift.

### Conclusioni

L'integrazione della libreria **ap\_shift\_reg** ha ottenuto eccellenti drisultati, stabilendo il record di minor periodo di clock stimato (2.686 ns) e riducendo l'impatto dei **DSP** al minimo

assoluto. Ciononostante, l'architettura rimane confinata entro un paradigma rigorosamente seriale che impedisce di abbattere l'Initiation Interval (II) al di sotto del collo di bottiglia dei colpi di clock iterativi. Visti gli straordinari risultati che la compressione dei dati ha dimostrato sulla variante compatta di hoisting, il prossimo passo prevederà un'inversione di rotta analitica: verrà tentata l'applicazione della **Bitwidth Optimization** direttamente sulla soluzione **02\_loop-fission**. Questo passaggio intermedio permetterà di verificare se il disaccoppiamento dei flussi logici (shift e MAC), unito alla contrazione dei tipi tramite **ap\_int**, sia in grado di generare una frontiera d'area e delay ancora più efficiente, ponendo le basi ideali per le successive trasformazioni di **parallelismo spaziale e temporale**.

#### 4.3.4. Esplorazione Combinata: «Loop Fission» con «Bitwidth Optimization»

Per completare il quadro delle ottimizzazioni a livello di sorgente sequenziale, in questo capitolo viene analizzata l'unione sistematica delle due strategie che finora hanno mostrato l'impatto più profondo sull'IP core. La soluzione **05\_loop-fission-ap-int** riprende direttamente il codice della variante a flussi disaccoppiati (Soluzione 02), in cui le fasi di aggiornamento della *buffer line* (**shift\_loop**) e di calcolo aritmetico (**mac\_loop**) sono state isolate in due cicli distinti, per applicarvi la **Bitwidth Optimization**. A livello implementativo, l'intervento è consistito nella sostituzione sistematica dei tipi nativi *C/C++* a 32 *bit* con i tipi a precisione arbitraria forniti dalla libreria `<ap_int.h>`. L'applicazione segue la medesima spiegata nella Sezione 4.3.1. In questo modo, il sintetizzatore HLS è stato forzato a generare due *datapath* non solo operativamente indipendenti, ma caratterizzati da bus e registri aritmetici compressi all'esatta dinamica dei segnali elaborati.

#### 4.3.5. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione **05\_loop-fission-ap-int** punta a massimizzare il risparmio di area combinatoria e aritmetica strutturale. Dividendo i flussi logici ed eliminando i bit ridondanti, ci aspettiamo che il sintetizzatore HLS riesca a spingere al massimo il riuso temporale dei componenti operativi più complessi (moltiplicatori). Sotto il profilo temporale, l'aspettativa hardware prevede un aumento inevitabile dei cicli di clock globali rispetto alla variante compatta di hoisting, poiché la *FSM* si trova a dover gestire l'esecuzione rigorosamente seriale di due cicli distinti, mantenendo però inalterata l'ottima risposta sul *Critical Path* ottenuta dal restringimento dei bus.

#### Timing

| Metric                       | 02 — Loop fission  | 05 — Loop fission +<br>ap_int | $\Delta$  |
|------------------------------|--------------------|-------------------------------|-----------|
| Target clock                 | 10.000 ns          | 10.000 ns                     | —         |
| Estimated clock / $F_{\max}$ | 6.210 ns           | 3.132 ns                      | ▼ 49.57 % |
| Max achievable $F_{\max}$    | <b>161.030 MHz</b> | <b>319.280 MHz</b>            | ▲ 98.27 % |
| Clock uncertainty            | 2.7 ns             | 2.7 ns                        | —         |

## Latency and Throughput

| Metric                              | 02 — Loop fission                                     | 05 — Loop fission + ap_int                            | $\Delta$                     |
|-------------------------------------|-------------------------------------------------------|-------------------------------------------------------|------------------------------|
| Latency max @ target clock          | 66cyc $\rightarrow$ 660.0 ns<br>(0.6600 $\mu$ s)      | 77cyc $\rightarrow$ 770.0 ns<br>(0.7700 $\mu$ s)      | $\blacktriangle$ 16.67 %     |
| Latency max @ estimated clock       | 66cyc $\rightarrow$ 409.9 ns<br>(0.4099 $\mu$ s)      | 77cyc $\rightarrow$ 241.2 ns<br>(0.2412 $\mu$ s)      | $\blacktriangledown$ 41.16 % |
| CoSim total exec. @ target clock    | 2009cyc $\rightarrow$ 20090.0 ns<br>(20.0900 $\mu$ s) | 2339cyc $\rightarrow$ 23390.0 ns<br>(23.3900 $\mu$ s) | $\blacktriangle$ 16.43 %     |
| CoSim total exec. @ estimated clock | 2009cyc $\rightarrow$ 12475.9 ns<br>(12.4759 $\mu$ s) | 2339cyc $\rightarrow$ 7325.7 ns<br>(7.3257 $\mu$ s)   | $\blacktriangledown$ 41.28 % |

## Resource Utilisation

| Metric                                                      | 02 — Loop fission | 05 — Loop fission + ap_int | $\Delta$                     |
|-------------------------------------------------------------|-------------------|----------------------------|------------------------------|
| LUT                                                         | 214 (0.40 %)      | 155 (0.29 %)               | $\blacktriangledown$ 27.57 % |
| FF                                                          | 199 (0.19 %)      | 80 (0.08 %)                | $\blacktriangledown$ 59.80 % |
| BRAM                                                        | 0 (0.00 %)        | 0 (0.00 %)                 | —                            |
| DSP                                                         | 2 (0.91 %)        | 1 (0.45 %)                 | $\blacktriangledown$ 50.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                   |                            | $\blacktriangledown$ 45.20 % |

## Power and Energy

| Metric                                           | 02 — Loop fission        | 05 — Loop fission + ap_int | $\Delta$                     |
|--------------------------------------------------|--------------------------|----------------------------|------------------------------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                    | = 0.00 %                     |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                    | = 0.00 %                     |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.104 W</b>             | = 0.00 %                     |
| Energy/op $E_{\text{op}}$                        | $4.263 \times 10^{-8}$ J | $2.508 \times 10^{-8}$ J   | $\blacktriangledown$ 41.16 % |
| Total energy $E_{\text{total}}$                  | $1.297 \times 10^{-6}$ J | $7.619 \times 10^{-7}$ J   | $\blacktriangledown$ 41.28 % |

## Figures of Merit

| Metric                   | 02 — Loop fission | 05 — Loop fission + ap_int | $\Delta$                     |
|--------------------------|-------------------|----------------------------|------------------------------|
| Area-Delay Product (ADP) | 186.9387          | 60.1517                    | $\blacktriangledown$ 67.82 % |



|                            |                         |                         |                  |
|----------------------------|-------------------------|-------------------------|------------------|
| Energy-Delay Product (EDP) | $1.6187 \times 10^{-2}$ | $5.5813 \times 10^{-3}$ | ▼ <b>65.52 %</b> |
|----------------------------|-------------------------|-------------------------|------------------|

#### Analisi delle metriche

Confrontando con la soluzione **02 – Loop fission** a precisione standard, emerge l’impatto della contrazione dei bit a parità di struttura logica. I **Flip-Flop** hardware crollano da 199 a soli 80 (un risparmio straordinario del 59.8%) e le LUT combinatorie scendono da 214 a 155 (una contrazione del 27.6%) grazie al forte snellimento delle reti di routing aritmetiche. Sul piano temporale, sebbene la latenza algoritmica aumenti da 66 a 77 *cicli* per via delle diverse scelte di scheduling del compilatore sui tipi ridotti, il periodo di clock stimato viene quasi dimezzato, passando da 6.21 ns a 3.132 ns (un abbattimento del 49.6%). Di conseguenza, il tempo reale complessivo di esecuzione dell’algoritmo crolla da 12476 ns a 7326 ns, registrando un guadagno netto del 41.3% sul **Delay** totale rispetto alla versione non ottimizzata nei tipi dati.

#### Timing

| Metric                             | 03 — Bitwidth opt (on v01) | 05 — Loop fission + ap_int | Δ        |
|------------------------------------|----------------------------|----------------------------|----------|
| Target clock                       | 10.000 ns                  | 10.000 ns                  | —        |
| Estimated clock / F <sub>max</sub> | 3.132 ns                   | 3.132 ns                   | = 0.00 % |
| Max achievable F <sub>max</sub>    | <b>319.280 MHz</b>         | <b>319.280 MHz</b>         | = 0.00 % |
| Clock uncertainty                  | 2.7 ns                     | 2.7 ns                     | —        |

#### Latency and Throughput

| Metric                              | 03 — Bitwidth opt (on v01)        | 05 — Loop fission + ap_int        | Δ                |
|-------------------------------------|-----------------------------------|-----------------------------------|------------------|
| Latency max @ target clock          | 54cyc → 540.0 ns (0.5400 μs)      | 77cyc → 770.0 ns (0.7700 μs)      | ▲ <b>42.59 %</b> |
| Latency max @ estimated clock       | 54cyc → 169.1 ns (0.1691 μs)      | 77cyc → 241.2 ns (0.2412 μs)      | ▲ <b>42.59 %</b> |
| CoSim total exec. @ target clock    | 1649cyc → 16490.0 ns (16.4900 μs) | 2339cyc → 23390.0 ns (23.3900 μs) | ▲ <b>41.84 %</b> |
| CoSim total exec. @ estimated clock | 1649cyc → 5164.7 ns (5.1647 μs)   | 2339cyc → 7325.7 ns (7.3257 μs)   | ▲ <b>41.84 %</b> |

#### Resource Utilisation

| Metric                                                      | 03 — Bitwidth opt<br>(on v01) | 05 — Loop fission +<br>ap_int | $\Delta$  |
|-------------------------------------------------------------|-------------------------------|-------------------------------|-----------|
| LUT                                                         | 120 (0.23 %)                  | 155 (0.29 %)                  | ▲ 29.17 % |
| FF                                                          | 77 (0.07 %)                   | 80 (0.08 %)                   | ▲ 3.90 %  |
| BRAM                                                        | 0 (0.00 %)                    | 0 (0.00 %)                    | —         |
| DSP                                                         | 2 (0.91 %)                    | 1 (0.45 %)                    | ▼ 50.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                               |                               | ▼ 31.97 % |

## Power and Energy

| Metric                                           | 03 — Bitwidth opt<br>(on v01) | 05 — Loop fission +<br>ap_int | $\Delta$  |
|--------------------------------------------------|-------------------------------|-------------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                       | 0.103 W                       | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                       | 0.001 W                       | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>                | <b>0.104 W</b>                | ▲ 0.97 %  |
| Energy/op $E_{\text{op}}$                        | $1.742 \times 10^{-8}$ J      | $2.508 \times 10^{-8}$ J      | ▲ 43.98 % |
| Total energy $E_{\text{total}}$                  | $5.320 \times 10^{-7}$ J      | $7.619 \times 10^{-7}$ J      | ▲ 43.22 % |

## Figures of Merit

| Metric                        | 03 — Bitwidth opt<br>(on v01) | 05 — Loop fission +<br>ap_int | $\Delta$   |
|-------------------------------|-------------------------------|-------------------------------|------------|
| Area-Delay Product<br>(ADP)   | 62.3375                       | 60.1517                       | ▼ 3.51 %   |
| Energy-Delay Product<br>(EDP) | $2.7474 \times 10^{-3}$       | $5.5813 \times 10^{-3}$       | ▲ 103.15 % |

### Analisi delle metriche

Il confronto metrico rispetto alla soluzione 03\_ap-int evidenzia un trade-off di area estremamente marcato. Dal punto di vista prestazionale, lo sdoppiamento del loop incrementa la latenza da 54 a **77 colpi di clock** (un incremento del 42.6%) ed innalza l'Initiation Interval (II) a 78, portando i cicli totali di co-simulazione a 2339. Questa penalizzazione temporale è la diretta conseguenza dell'overhead sequenziale richiesto per completare lo `shift_loop` prima di poter avviare il `mac_loop`. Tuttavia, sul fronte delle risorse fisiche, questa architettura consegue un traguardo eccezionale: l'impiego dei blocchi **DSP** viene dimezzato, scendendo da 2 a un solo 1 blocco istanziato. Disaccoppiare il ciclo aritmetico ha infatti permesso al sintetizzatore di mappare tutte le 11 moltiplicazioni su un'unica unità MAC condivisa nel tempo. I **Flip-Flop** rimangono pressoché invariati (80 contro 77, +3.9%) mentre le LUT combinatorie registrano un incremento fisiologico da 120 a 155 (un aumento

del 29.2%) dovuto alla logica di controllo extra per la gestione delle transizioni di stato della nuova *FSM*. Il periodo di clock stimato si conserva perfettamente stabile a 3.132 ns.

## Conclusioni

La *Reef* fissione e la combinazione di **Loop Fission** e **Bitwidth Optimization** ha dimostrato come lo smantellamento delle dipendenze all'interno del loop principale sblocchi una straordinaria efficienza d'area, abbattendo l'uso dei **DSP** al minimo e preservando un ottimo *Critical Path*. Con questa analisi si conclude la fase di ottimizzazione del codice puramente sequenziale: le soluzioni **04\_ap-shift-reg** (ottimizzazione della memoria tramite primitive SRL) e **05\_loop-fission-ap-int** (ottimizzazione dei flussi e dei tipi dati) rappresentano i due migliori pilastri di partenza ottenuti nel workflow.

Si fa notare che l'applicazione della libreria *ap\_shift\_reg* sulla presente soluzione **05** è stata esplicitamente omessa dal piano di azione, in quanto l'integrazione di tale costrutto avrebbe rimosso lo shift manuale degenerando l'architettura all'interno della stessa soluzione **04**. Di conseguenza, i passi successivi prevederanno l'applicazione diretta e parallela delle tecniche di parallelismo spaziale e temporale (**Loop Unrolling** e **Loop Pipelining**, assistiti da **Array Partitioning**) separatamente sulle basi architetturali stabili delle varianti **04** e **05**, per determinare quale configurazione permetterà di abbattere il vincolo sequenziale e raggiungere il throughput ideale di 1 colpo di clock.

## 4.4. Parallelismo

### 4.4.1. Parallelismo Spaziale Iniziale: Unrolling dello «shift\_loop»

#### 4.4.1.1. Descrizione dell'Implementazione

L'esplorazione del parallelismo spaziale ha inizio isolando il blocco di codice deputato all'aggiornamento della linea di ritardo. Nella soluzione **06\_loop-fission-unroll**, l'intervento si concentra esclusivamente sul ciclo *shift\_loop* ereditato dall'architettura a flussi disaccoppiati della soluzione **05**. A livello di codice, è stata inserita la direttiva `#pragma HLS unroll` immediatamente all'interno del corpo del primo *loop*:

```
...

// --- Shift REG ---
shift_loop: for (int i = N-1; i > 0; i--) {
    #pragma HLS unroll /* OPTIMIZATION(unroll): unrolling the shift loop

    shift_reg[i] = shift_reg[i - 1]; // Shift i-1 element to the right (drop the
last element)
}

...
```

Codice 8: Applicazione del **Loop Unrolling** completo sul ciclo di *shift* della *FIFO*.

Questa modifica istruisce il sintetizzatore HLS a distendere completamente le iterazioni del ciclo, rimpiazzando la struttura di controllo iterativa con  $10^5$  assegnamenti fisici paralleli eseguiti in logica combinatoria. Il ciclo aritmetico *mac\_loop* viene invece mantenuto temporaneamente

inalterato (rollato e sequenziale) per isolare l'impatto di questa prima trasformazione spaziale sul datapath globale dell'IP core.

#### 4.4.1.2. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione **06\_loop-fission-unroll** ha l'obiettivo di abbattere l'overhead temporale della linea di ritardo linearizzando lo scorrimento dei campioni. Ci aspettiamo che la rimozione dei vincoli iterativi sullo *shift* riduca la latenza complessiva dell'algoritmo di un fattore pari al numero di tap del filtro. Tuttavia, l'efficacia di questa operazione a livello hardware è strettamente subordinata alle limitazioni di accesso alle porte della memoria RAM statica, che potrebbero indurre colli di bottiglia strutturali a causa dei molteplici accessi simultanei richiesti dai registri paralleli.

##### Timing

| Metric                       | 05 — Loop fission +<br>ap_int | 06 — Loop unroll<br>(shift) | $\Delta$  |
|------------------------------|-------------------------------|-----------------------------|-----------|
| Target clock                 | 10.000 ns                     | 10.000 ns                   | —         |
| Estimated clock / $F_{\max}$ | 3.132 ns                      | 2.686 ns                    | ▼ 14.24 % |
| Max achievable $F_{\max}$    | <b>319.280 MHz</b>            | <b>372.300 MHz</b>          | ▲ 16.61 % |
| Clock uncertainty            | 2.7 ns                        | 2.7 ns                      | —         |

##### Latency and Throughput

| Metric                              | 05 — Loop fission +<br>ap_int             | 06 — Loop unroll<br>(shift)               | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 77cyc → 770.0 ns<br>(0.7700 $\mu$ s)      | 65cyc → 650.0 ns<br>(0.6500 $\mu$ s)      | ▼ 15.58 % |
| Latency max @ estimated clock       | 77cyc → 241.2 ns<br>(0.2412 $\mu$ s)      | 65cyc → 174.6 ns<br>(0.1746 $\mu$ s)      | ▼ 27.61 % |
| CoSim total exec. @ target clock    | 2339cyc → 23390.0 ns<br>(23.3900 $\mu$ s) | 1979cyc → 19790.0 ns<br>(19.7900 $\mu$ s) | ▼ 15.39 % |
| CoSim total exec. @ estimated clock | 2339cyc → 7325.7 ns<br>(7.3257 $\mu$ s)   | 1979cyc → 5315.6 ns<br>(5.3156 $\mu$ s)   | ▼ 27.44 % |

##### Resource Utilisation

| Metric | 05 — Loop fission +<br>ap_int | 06 — Loop unroll<br>(shift) | $\Delta$   |
|--------|-------------------------------|-----------------------------|------------|
| LUT    | 155 (0.29 %)                  | 218 (0.41 %)                | ▲ 40.65 %  |
| FF     | 80 (0.08 %)                   | 175 (0.16 %)                | ▲ 118.75 % |

|                                                             |            |            |           |
|-------------------------------------------------------------|------------|------------|-----------|
| BRAM                                                        | 0 (0.00 %) | 0 (0.00 %) | —         |
| DSP                                                         | 1 (0.45 %) | 1 (0.45 %) | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |            |            | ▲ 25.30 % |

## Power and Energy

| Metric                                           | 05 — Loop fission +<br>ap_int | 06 — Loop unroll<br>(shift) | $\Delta$  |
|--------------------------------------------------|-------------------------------|-----------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                       | 0.103 W                     | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                       | 0.001 W                     | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>                | <b>0.104 W</b>              | = 0.00 %  |
| Energy/op $E_{\text{op}}$                        | $2.508 \times 10^{-8}$ J      | $1.816 \times 10^{-8}$ J    | ▼ 27.61 % |
| Total energy $E_{\text{total}}$                  | $7.619 \times 10^{-7}$ J      | $5.528 \times 10^{-7}$ J    | ▼ 27.44 % |

## Figures of Merit

| Metric                        | 05 — Loop fission +<br>ap_int | 06 — Loop unroll<br>(shift) | $\Delta$  |
|-------------------------------|-------------------------------|-----------------------------|-----------|
| Area-Delay Product<br>(ADP)   | 60.1517                       | 54.6868                     | ▼ 9.09 %  |
| Energy-Delay Product<br>(EDP) | $5.5813 \times 10^{-3}$       | $2.9386 \times 10^{-3}$     | ▼ 47.35 % |

### Analisi delle metriche

L'analisi delle metriche convalida parzialmente i benefici della parallelizzazione del controllo. Sotto il profilo temporale, la latenza di calcolo per singolo campione scende da 77 a 65 **colpi di clock** (un risparmio netto del 15.6%), provocando una contrazione proporzionale dell'Initiation Interval (II) che passa a 66 cicli e riducendo i *cicli totali* di co-simulazione a 1979. Questo incremento prestazionale si accompagna a un marcato rilassamento del *Critical Path*: il periodo di clock stimato si contrae da 3.132 ns a soli 2.686 ns (speedup del 14.2%). Di conseguenza, il tempo assoluto di elaborazione reale sperimenta un crollo netto, scendendo a 174.59 ns rispetto ai 241.16 ns della soluzione 05 (guadagno del 27.6%). Questo balzo prestazionale si scontra però con un incremento nell'occupazione di area: le LUT combinatorie aumentano da 155 a 218 (un incremento del 40.6%) e i Flip-Flop salgono da 80 a 175 (aumento del 118.7%). Tale espansione fisica è la conseguenza diretta della replicazione hardware dei bus di indirizzamento e dei registri di supporto necessari per mappare lo *shift* parallelo su una memoria non ancora ottimizzata nei canali di accesso. L'impiego dei blocchi **DSP** si mantiene stabile a 1.

## Conclusioni

L'applicazione del **Loop Unrolling** sullo `shift_loop` ha confermato l'abbattimento dei tempi morti della **FSM**, ottimizzando significativamente il *Delay* reale e la frequenza massima operativa. Tuttavia, l'analisi del profilo di sintesi rivela che il sintetizzatore è stato limitato nel parallelismo concorrente: l'array `shift_reg` risiede ancora in una struttura di memoria standard a porte limitate, generando una contesa strutturale sui canali di lettura e scrittura che frena le massime prestazioni potenziali dell'unrolling. Per rimuovere questo collo di bottiglia strutturale e permettere un accesso simultaneo e illimitato a tutti i nodi della linea di ritardo, il passo successivo consisterà nell'integrazione della direttiva di **Array Partitioning** combinata per completare la totale scomposizione hardware dei registri di stato.

### 4.4.2. Risoluzione dei Colli di Bottiglia della Memoria: Completamento con «Array Partitioning»

#### 4.4.2.1. Descrizione dell'Implementazione

Avendo riscontrato limitazioni fisiche nell'accesso concorrente ai dati all'interno della memoria di stato, la soluzione `07_loop-fission-array-partition` introduce un intervento mirato sulla struttura di memorizzazione dell'array. L'ottimizzazione è stata applicata inserendo la direttiva `#pragma HLS array_partition` associata alla variabile statica `shift_reg` con opzione di partizionamento di tipo `complete`:

```
...

// Static shift register maintains data persistence across multiple function
calls
static in_data_t shift_reg[N] = {0}; // Initialize with zeros
#pragma HLS array_partition variable=shift_reg complete /* OPTIMIZATION(array-
partition): partitioning the shift register array completely

...
```

Codice 9: Mappatura hardware ottimizzata tramite la direttiva di **Array Partitioning** sull'array della FIFO.

A livello implementativo, questa configurazione impone al compilatore HLS di frantumare l'array statico originario, rimpiazzando l'allocazione in memoria RAM con  $\$11\$$  registri hardware fisici, discreti e indipendenti. Ciascun elemento della linea di ritardo viene così equipaggiato con un proprio datapath di lettura e scrittura dedicato, eliminando alla radice l'overhead dei multiplexer di indirizzamento dinamico e sbloccando la piena esecuzione concorrente delle istruzioni precedentemente linearizzate dall'unrolling dello `shift_loop`.

#### 4.4.2.2. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione `07_loop-fission-array-partition` agisce come elemento abilitante per l'ottimizzazione spaziale. Distruggendo la struttura a memoria centralizzata dell'array a favore di registri individuali, l'aspettativa hardware risiede nella totale eliminazione dei conflitti di porta fisici durante le transizioni simultanee dello *shift register*. Ci aspettiamo un crollo immediato dei cicli di clock richiesti per completare la linea di ritardo e, contem-

poraneamente, una decisa contrazione delle risorse logiche combinatorie (LUT), liberate dall'infrastruttura di commutazione e indirizzamento dei vecchi canali di memoria.

## Timing

| Metric                       | 06 — Loop unroll (shift) |                    | $\Delta$ |
|------------------------------|--------------------------|--------------------|----------|
| Target clock                 | 10.000 ns                | 10.000 ns          | —        |
| Estimated clock / $F_{\max}$ | 2.686 ns                 | 2.686 ns           | = 0.00 % |
| Max achievable $F_{\max}$    | <b>372.300 MHz</b>       | <b>372.300 MHz</b> | = 0.00 % |
| Clock uncertainty            | 2.7 ns                   | 2.7 ns             | —        |

## Latency and Throughput

| Metric                              | 06 — Loop unroll (shift)                  |                                           | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 65cyc → 650.0 ns<br>(0.6500 $\mu$ s)      | 56cyc → 560.0 ns<br>(0.5600 $\mu$ s)      | ▼ 13.85 % |
| Latency max @ estimated clock       | 65cyc → 174.6 ns<br>(0.1746 $\mu$ s)      | 56cyc → 150.4 ns<br>(0.1504 $\mu$ s)      | ▼ 13.85 % |
| CoSim total exec. @ target clock    | 1979cyc → 19790.0 ns<br>(19.7900 $\mu$ s) | 1709cyc → 17090.0 ns<br>(17.0900 $\mu$ s) | ▼ 13.64 % |
| CoSim total exec. @ estimated clock | 1979cyc → 5315.6 ns<br>(5.3156 $\mu$ s)   | 1709cyc → 4590.4 ns<br>(4.5904 $\mu$ s)   | ▼ 13.64 % |

## Resource Utilisation

| Metric                                                      | 06 — Loop unroll (shift) |              | $\Delta$  |
|-------------------------------------------------------------|--------------------------|--------------|-----------|
| LUT                                                         | 218 (0.41 %)             | 120 (0.23 %) | ▼ 44.95 % |
| FF                                                          | 175 (0.16 %)             | 310 (0.29 %) | ▲ 77.14 % |
| BRAM                                                        | 0 (0.00 %)               | 0 (0.00 %)   | —         |
| DSP                                                         | 1 (0.45 %)               | 1 (0.45 %)   | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                          |              | ▼ 5.57 %  |

## Power and Energy

| Metric | 06 — Loop unroll (shift) |  | $\Delta$ |
|--------|--------------------------|--|----------|
|--------|--------------------------|--|----------|

|                                                  |                          |                          |            |
|--------------------------------------------------|--------------------------|--------------------------|------------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = 0.00 %   |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.002 W                  | ▲ 100.00 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.105 W</b>           | ▲ 0.96 %   |
| Energy/op $E_{\text{op}}$                        | $1.816 \times 10^{-8}$ J | $1.579 \times 10^{-8}$ J | ▼ 13.01 %  |
| Total energy $E_{\text{total}}$                  | $5.528 \times 10^{-7}$ J | $4.820 \times 10^{-7}$ J | ▼ 12.81 %  |

## Figures of Merit

| Metric                     | 06 — Loop unroll (shift) |                         | $\Delta$  |
|----------------------------|--------------------------|-------------------------|-----------|
| Area-Delay Product (ADP)   | 54.6868                  | 44.5955                 | ▼ 18.45 % |
| Energy-Delay Product (EDP) | $2.9386 \times 10^{-3}$  | $2.2125 \times 10^{-3}$ | ▼ 24.71 % |

### Analisi delle metriche

I risultati estratti dai report confermano uno scenario di ottimizzazione straordinario. Sotto il profilo temporale, il superamento del collo di bottiglia della memoria permette alla latenza algoritmica di contrarsi ulteriormente da 65 a 56 **colpi di clock** (un guadagno netto del 13.8%), stabilizzando l'Initiation Interval (II) a 57 cicli e riducendo il tempo di esecuzione totale della co-simulazione a 1709 cicli. Il periodo di clock stimato si consolida all'ottimo valore di 2.686 ns, portando l'effettivo **Delay** operativo reale dell'IP core al record parziale di soli 150.42 ns. L'aspetto più dirompente si registra tuttavia sul fronte d'area hardware. La completa rimozione della logica di multiplexing dinamico necessaria per indirizzare l'array centralizzato fa letteralmente crollare l'uso delle LUT combinatorie da 218 a sole 120 (una riduzione netta del 44.9%), ottimizzando sensibilmente il routing combinatorio. Di contro, la scomposizione fisica dell'array in celle discrete si paga con un incremento atteso dei Flip-Flop, che salgono da 175 a 310 (un aumento del 77.1%) a causa dell'allocazione dei singoli registri hardware indipendenti per ogni tap del filtro. L'impiego dei blocchi **DSP** rimane ancorato al minimo di 1.

### Conclusioni

L'integrazione combinata di **Loop Unrolling** e **Array Partitioning** sulla linea di ritardo ha dimostrato come la rimozione dei colli di bottiglia fisici della memoria RAM possa contemporaneamente incrementare il throughput temporale assoluto e abbattere drasticamente l'area logica combinatoria (LUT) dell'IP core. L'architettura hardware così ottenuta rappresenta la massima ottimizzazione spaziale realizzabile sulla gestione dello stato sequenziale. Avendo consolidato ed esaurito i margini di miglioramento sul datapath puramente spaziale, il passo successivo del piano di azione consisterà nell'introdurre il parallelismo temporale attraverso l'applicazione della direttiva di **Loop Pipelining** direttamente sulla configurazione sequenziale compatta della soluzione 05. Questa analisi parallela permetterà



di valutare se la sovrapposizione temporale delle fasi di calcolo possa scavalcare definitivamente l'overhead dei cicli iterativi, puntando al target prestazionale ideale di un Initiation Interval pari a 1\$ colpo di clock.

#### 4.4.3. Parallelismo Temporale Iniziale: Pipelining del «mac\_loop»

##### 4.4.3.1. Descrizione dell'Implementazione

L'esplorazione dell'ottimizzazione concorrente prosegue introducendo il parallelismo temporale sul datapath di calcolo numerico. Nella soluzione `08_loop-fission-pipeline`, l'intervento si concentra sull'ottimizzazione del ciclo di accumulo aritmetico, lasciando inalterata la struttura dei restanti blocchi logici. A livello di codice, l'intervento è consistito nell'inserimento della direttiva `#pragma HLS pipeline II=1` immediatamente all'interno del corpo del `mac_loop`, applicandola direttamente sulla variante a flussi disaccoppiati e ridotti nei tipi dati ereditata dalla soluzione 05 (**Loop Fission + Bitwidth Optimization**):

```
...

// --- Shift REG ---
shift_loop: for (int i = N-1; i > 0; i--) {
    shift_reg[i] = shift_reg[i - 1];
}
/* OPTIMIZATION(code-hoisting): removed conditional check in the loop
shift_reg[0] = x; // Insert the new data sample at the beginning

// --- MAC ---
mac_loop: for (int i = 0; i < N; i++) {
    #pragma HLS pipeline II=1 /* OPTIMIZATION(pipeline): executing MAC
operations in pipeline

    acc += shift_reg[i] * c[i]; // Perform MAC on the i-th element
}

...
```

Codice 10: Applicazione del **Loop Pipelining** con target `II=1` sul ciclo d'accumulo aritmetico.

Questa direttiva impone al sintetizzatore HLS di inserire registri di sfasamento hardware controllati all'interno della rete combinatoria di moltiplicazione e addizione. In questo modo, l'hardware viene predisposto per l'esecuzione in sovrapposizione temporale delle iterazioni successive dell'algoritmo MAC: il circuito è in grado di accettare un nuovo operando e far avanzare lo stadio della pipeline ad ogni colpo di clock, puntando a un **Initiation Interval** (II) interno al loop pari a 1. Lo `shift_loop` viene invece mantenuto interamente rollato e sequenziale per isolare analiticamente gli effetti del pipelining temporale isolato.

#### 4.4.4. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione `08_loop-fission-pipeline` persegue l'abbattimento dei tempi morti dell'unità aritmetica tramite la sovrapposizione temporale delle fasi d'esecuzione. Ci aspettiamo che l'introduzione della pipeline nel `mac_loop` riduca drasticamente il numero di cicli di clock necessari per esaurire l'accumulo dei tap del filtro. Tuttavia, la persistenza di un ciclo di aggiornamento dello shift register statico completamente seriale, unito all'uso di un array

non ancora partizionato, potrebbe forzare il sintetizzatore HLS a un scheduling rigido, introducendo un severo overhead combinatorio sul circuito risultante.

## Timing

| Metric                       | 05 — Loop fission + ap_int | 08 — Loop Pipeline (MAC) | $\Delta$  |
|------------------------------|----------------------------|--------------------------|-----------|
| Target clock                 | 10.000 ns                  | 10.000 ns                | —         |
| Estimated clock / $F_{\max}$ | 3.132 ns                   | 4.226 ns                 | ▲ 34.93 % |
| Max achievable $F_{\max}$    | <b>319.280 MHz</b>         | <b>236.630 MHz</b>       | ▼ 25.89 % |
| Clock uncertainty            | 2.7 ns                     | 2.7 ns                   | —         |

## Latency and Throughput

| Metric                              | 05 — Loop fission + ap_int                | 08 — Loop Pipeline (MAC)                  | $\Delta$  |
|-------------------------------------|-------------------------------------------|-------------------------------------------|-----------|
| Latency max @ target clock          | 77cyc → 770.0 ns<br>(0.7700 $\mu$ s)      | 40cyc → 380.0 ns<br>(0.3800 $\mu$ s)      | ▼ 50.65 % |
| Latency max @ estimated clock       | 77cyc → 241.2 ns<br>(0.2412 $\mu$ s)      | 40cyc → 160.6 ns<br>(0.1606 $\mu$ s)      | ▼ 33.41 % |
| CoSim total exec. @ target clock    | 2339cyc → 23390.0 ns<br>(23.3900 $\mu$ s) | 1169cyc → 11690.0 ns<br>(11.6900 $\mu$ s) | ▼ 50.02 % |
| CoSim total exec. @ estimated clock | 2339cyc → 7325.7 ns<br>(7.3257 $\mu$ s)   | 1169cyc → 4940.2 ns<br>(4.9402 $\mu$ s)   | ▼ 32.56 % |

## Resource Utilisation

| Metric                                                      | 05 — Loop fission + ap_int | 08 — Loop Pipeline (MAC) | $\Delta$  |
|-------------------------------------------------------------|----------------------------|--------------------------|-----------|
| LUT                                                         | 155 (0.29 %)               | 231 (0.43 %)             | ▲ 49.03 % |
| FF                                                          | 80 (0.08 %)                | 153 (0.14 %)             | ▲ 91.25 % |
| BRAM                                                        | 0 (0.00 %)                 | 0 (0.00 %)               | —         |
| DSP                                                         | 1 (0.45 %)                 | 1 (0.45 %)               | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                            |                          | ▲ 25.76 % |

## Power and Energy

| Metric | 05 — Loop fission + ap_int | 08 — Loop Pipeline (MAC) | $\Delta$ |
|--------|----------------------------|--------------------------|----------|
|--------|----------------------------|--------------------------|----------|

|                                                  |                          |                          |           |
|--------------------------------------------------|--------------------------|--------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                  | = 0.00 %  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.104 W</b>           | <b>0.104 W</b>           | = 0.00 %  |
| Energy/op $E_{\text{op}}$                        | $2.508 \times 10^{-8}$ J | $1.670 \times 10^{-8}$ J | ▼ 33.41 % |
| Total energy $E_{\text{total}}$                  | $7.619 \times 10^{-7}$ J | $5.138 \times 10^{-7}$ J | ▼ 32.56 % |

## Figures of Merit

| Metric                     | 05 — Loop fission + ap_int | 08 — Loop Pipeline (MAC) | $\Delta$  |
|----------------------------|----------------------------|--------------------------|-----------|
| Area-Delay Product (ADP)   | 60.1517                    | 51.0124                  | ▼ 15.19 % |
| Energy-Delay Product (EDP) | $5.5813 \times 10^{-3}$    | $2.5382 \times 10^{-3}$  | ▼ 54.52 % |

### Analisi delle metriche

Il confronto metrico con la soluzione sequenziale a bit ridotti (05\_loop-fission-ap-int) convalida l'efficacia del pipelining sul fronte dei cicli di clock, mostrando però un sensibile costo hardware. Dal punto di vista temporale, la latenza algoritmica complessiva della funzione quasi dimezza, crollando da 77 a soli 40 **colpi di clock** (un risparmio netto del 48.1%), provocando un decremento speculare dei **cicli totali** di esecuzione in co-simulazione, che scendono da 2339 a 1169 (riduzione del 50%). Questo netto incremento del throughput si scontra tuttavia con un marcato degrado del **periodo di clock stimato**, che si innalza da 3.132 ns a 4.226 ns (un incremento del 34.9% del **Critical Path**). L'infrastruttura logica richiesta per governare l'avanzamento dei dati in pipeline e le reti di instradamento necessarie per mappare le iterazioni sovrapposte su un'unica unità aritmetica condivisa (l'impiego dei blocchi **DSP** rimane ancorato a un solo 1) appesantiscono i ritardi combinatori. Questo si riflette anche sulle risorse fisiche: le LUT combinatorie salgono da 155 a 231 (aumento del 49%) e i Flip-Flop passano da 80 a 153 (un incremento del 91.25%) a causa dell'istanza dei registri di sfasamento interni alla pipeline.

## Timing

| Metric                             | 07 — Array Partitioning | 08 — Loop Pipeline (MAC) | $\Delta$  |
|------------------------------------|-------------------------|--------------------------|-----------|
| Target clock                       | 10.000 ns               | 10.000 ns                | —         |
| Estimated clock / $F_{\text{max}}$ | 2.686 ns                | 4.226 ns                 | ▲ 57.33 % |
| Max achievable $F_{\text{max}}$    | <b>372.300 MHz</b>      | <b>236.630 MHz</b>       | ▼ 36.44 % |
| Clock uncertainty                  | 2.7 ns                  | 2.7 ns                   | —         |

## Latency and Throughput

| Metric                              | 07 — Array Partitioning                               | 08 — Loop Pipeline (MAC)                              | $\Delta$  |
|-------------------------------------|-------------------------------------------------------|-------------------------------------------------------|-----------|
| Latency max @ target clock          | 56cyc $\rightarrow$ 560.0 ns<br>(0.5600 $\mu$ s)      | 40cyc $\rightarrow$ 380.0 ns<br>(0.3800 $\mu$ s)      | ▼ 32.14 % |
| Latency max @ estimated clock       | 56cyc $\rightarrow$ 150.4 ns<br>(0.1504 $\mu$ s)      | 40cyc $\rightarrow$ 160.6 ns<br>(0.1606 $\mu$ s)      | ▲ 6.76 %  |
| CoSim total exec. @ target clock    | 1709cyc $\rightarrow$ 17090.0 ns<br>(17.0900 $\mu$ s) | 1169cyc $\rightarrow$ 11690.0 ns<br>(11.6900 $\mu$ s) | ▼ 31.60 % |
| CoSim total exec. @ estimated clock | 1709cyc $\rightarrow$ 4590.4 ns<br>(4.5904 $\mu$ s)   | 1169cyc $\rightarrow$ 4940.2 ns<br>(4.9402 $\mu$ s)   | ▲ 7.62 %  |

## Resource Utilisation

| Metric                                                      | 07 — Array Partitioning | 08 — Loop Pipeline (MAC) | $\Delta$  |
|-------------------------------------------------------------|-------------------------|--------------------------|-----------|
| LUT                                                         | 120 (0.23 %)            | 231 (0.43 %)             | ▲ 92.50 % |
| FF                                                          | 310 (0.29 %)            | 153 (0.14 %)             | ▼ 50.65 % |
| BRAM                                                        | 0 (0.00 %)              | 0 (0.00 %)               | —         |
| DSP                                                         | 1 (0.45 %)              | 1 (0.45 %)               | = 0.00 %  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                         |                          | ▲ 6.29 %  |

## Power and Energy

| Metric                                           | 07 — Array Partitioning  | 08 — Loop Pipeline (MAC) | $\Delta$  |
|--------------------------------------------------|--------------------------|--------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.002 W                  | 0.001 W                  | ▼ 50.00 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.105 W</b>           | <b>0.104 W</b>           | ▼ 0.95 %  |
| Energy/op $E_{\text{op}}$                        | $1.579 \times 10^{-8}$ J | $1.670 \times 10^{-8}$ J | ▲ 5.74 %  |
| Total energy $E_{\text{total}}$                  | $4.820 \times 10^{-7}$ J | $5.138 \times 10^{-7}$ J | ▲ 6.60 %  |

## Figures of Merit

| Metric                   | 07 — Array Partitioning | 08 — Loop Pipeline (MAC) | $\Delta$  |
|--------------------------|-------------------------|--------------------------|-----------|
| Area-Delay Product (ADP) | 44.5955                 | 51.0124                  | ▲ 14.39 % |

|                            |                         |                         |           |
|----------------------------|-------------------------|-------------------------|-----------|
| Energy-Delay Product (EDP) | $2.2125 \times 10^{-3}$ | $2.5382 \times 10^{-3}$ | ▲ 14.72 % |
|----------------------------|-------------------------|-------------------------|-----------|

#### Analisi delle metriche

Estendendo il confronto alla variante a parallelismo spaziale massimo (07 - Array Partitioning), si delinea un trade-off architetturale di estremo rilievo tra area logica e tempo assoluto di calcolo. Sotto il profilo dei cicli d'orologio puri, la pipeline temporale della soluzione 08 si dimostra superiore alla linearizzazione dello shift spaziale, esibendo una latenza di soli 40 cicli contro i 56 della soluzione 07 (un guadagno del 28.6% in cicli). Tuttavia, se traduciamo questo dato in tempo reale assoluto moltiplicando i cicli per il clock stimato, l'effettivo **Delay** operativo della soluzione 08 ( $40 \times 4.226 = 169.04$  ns) risulta più lento rispetto ai 150.42 ns della soluzione 07, la quale beneficiava di un cammino critico eccezionalmente rilassato a 2.686 ns. Sotto il profilo dell'occupazione delle risorse hardware sul silicio, la soluzione 08 preserva una notevole compattezza sul piano dei Flip-Flop, richiedendone solo 153 contro i 310 della soluzione 07 (un risparmio del 50.6%), poiché evita di frantumare fisicamente l'array in registri discreti. Di contro, la necessità di indirizzare dinamicamente la memoria RAM rolled centralizzata per alimentare il datapath pipelinato fa esplodere l'uso delle LUT combinatorie, che salgono a 231 contro le sole 120 della soluzione 07, dove l'assenza di strutture di memoria centralizzate azzerava l'overhead delle reti di multiplexing.

#### Conclusioni

L'analisi della soluzione con **Loop Pipelining** isolato sul blocco aritmetico dimostra come la sovrapposizione temporale sia la strategia più incisiva per abbattere i cicli d'orologio sprecati dalla **FSM**. Al contempo, emerge chiaramente una severa limitazione strutturale: forzare l'esecuzione pipelinata condividendo nel tempo un unico blocco DSP su un array rolled non partizionato satura le interconnessioni combinatorie, degradando il **Critical Path** dell'IP core e innalzando il periodo di clock stimato a 4.226 ns.

Avendo consolidato l'esplorazione indipendente del parallelismo spaziale puro (Soluzione 07, caratterizzata dal minimo clock e ridotta area combinatoria) e del parallelismo temporale isolato (Soluzione 08, caratterizzata dal minor numero di cicli complessivi), la metodologia di progetto impone la convergenza sinergica delle due tecniche. Di conseguenza, i prossimi passi del workflow prevederanno di riprendere la solida e sbloccata struttura architetturale della soluzione 07\_loop-fission-array-partition per estendere l'applicazione del parallelismo temporale e spaziale anche sul ciclo del **MAC**. L'introduzione coordinata di pipeline e unrolling sul ciclo aritmetico sfrutterà la memoria di stato già completamente partizionata per eliminare alla radice ogni conflitto d'accesso, abbattendo contemporaneamente i cicli di clock e i ritardi combinatori per traguardare finalmente l'obiettivo supremo di un **Initiation Interval** (II) globale pari a 1 colpo di clock.

#### 4.4.5. Convergenza di Strategie: Parallelizzazione Avanzata del Ciclo Aritmetico

In questa fase del workflow il focus si sposta dall'ottimizzazione esclusiva dello storage sequenziale all'introduzione del parallelismo concorrente sul datapath aritmetico. Avendo rimosso i

colli di bottiglia fisici della memoria RAM tramite la scomposizione completa operata nella soluzione 07, è adesso possibile esplorare l'applicazione combinata di tecniche di parallelismo spaziale e temporale sul ciclo di Multiply-Accumulate (MAC). Vengono analizzate di seguito tre distinte architetture di ottimizzazione applicate al ciclo aritmetico per determinare il miglior bilanciamento tra throughput operativo e occupazione di area sul silicio.

#### 4.4.5.1. Soluzione 09: Loop Pipelining sul Ciclo MAC con Storage Partizionato

##### 4.4.5.1.1. Descrizione dell'Implementazione

La soluzione `09_loop-fission-unroll-pipeline` introduce il parallelismo temporale direttamente sulla struttura ottimizzata e sbloccata della soluzione 07. A livello implementativo, l'intervento consiste nell'inserimento della direttiva `#pragma HLS pipeline II=1` all'interno del corpo del `mac_loop`. Questa configurazione opera in perfetta sinergia con il partizionamento completo dell'array `shift_reg` e lo srotolamento totale dello `shift_loop`:

```
...

// --- Shift REG ---
shift_loop: for (int i = N-1; i > 0; i--) {
    #pragma HLS unroll
    shift_reg[i] = shift_reg[i - 1];
}
shift_reg[0] = x;

// --- MAC ---
mac_loop: for (int i = 0; i < N; i++) {
    #pragma HLS pipeline II=1 /* OPTIMIZATION(pipeline): pipeline the MAC loop
with II=1
    acc += shift_reg[i] * c[i];
}

...
```

Codice 11: Applicazione del **Loop Pipelining** concorrente sul ciclo aritmetico combinato con l'unrolling dello shift.

Grazie alla scomposizione dell'array in registri discreti, il sintetizzatore HLS è in grado di schedulare le letture simultanee di tutti i campioni storici senza generare conflitti strutturali sulle porte di memoria. I registri di sfasamento della pipeline vengono allocati lungo la rete combinatoria per consentire la sovrapposizione temporale delle iterazioni del MAC, permettendo all'unità di accettare un nuovo operando ad ogni colpo di orologio.

#### 4.4.5.2. Soluzione 10: Loop Unrolling Totale del Ciclo MAC

##### 4.4.5.2.1. Descrizione dell'Implementazione

La soluzione `10_total-unroll` persegue la via del parallelismo spaziale estremo eliminando completamente la natura iterativa del calcolo. L'ottimizzazione viene eseguita rimpiazzando la pipeline temporale con la direttiva `#pragma HLS unroll` applicata in modo totale sul ciclo `mac_loop`:

```

...

// --- MAC ---
mac_loop: for (int i = 0; i < N; i++) {
    #pragma HLS unroll /* OPTIMIZATION(unroll): completely unroll the MAC loop
    acc += shift_reg[i] * c[i];
}

```

Codice 12: Srotolamento completo (**Total Unrolling**) del ciclo aritmetico per la generazione di una rete hardware parallela.

Questa trasformazione costringe il compilatore HLS a distendere l'algoritmo in un'unica rete combinatoria piana composta da 11 moltiplicatori fisici operanti in parallelo, le cui uscite convergono verso un albero di sommatore. Poiché ogni tap del filtro dispone di una risorsa di calcolo dedicata e indipendente, la sottomissione dei dati avviene in un unico passo hardware, portando la latenza iterativa interna al ciclo al minimo teorico.

#### 4.4.5.3. Soluzione 11: Loop Unrolling Parziale del Ciclo MAC

##### 4.4.5.3.1. Descrizione dell'Implementazione

La soluzione `11_partial-unroll` introduce una strategia di compromesso spaziale per mitigare l'elevato consumo di risorse indotto dalla linearizzazione totale. L'implementazione si basa sull'applicazione della direttiva `#pragma HLS unroll` configurata con un fattore di srotolamento controllato, definito tramite l'espressione `factor=N/2` (che corrisponde a un fattore intero pari a 5):

```

...

// --- MAC ---
mac_loop: for (int i = 0; i < N; i++) {
    #pragma HLS unroll factor=N/2 /* OPTIMIZATION(unroll): partially unroll the
MAC loop by a factor of 5
    acc += shift_reg[i] * c[i];
}

```

...

Codice 13: Configurazione del **Loop Unrolling Parziale** per il bilanciamento hardware del datapath aritmetico.

A livello hardware, questa direttiva impone una scomposizione parziale del ciclo: la **FSM** viene configurata per gestire un datapath parallelo ristretto a 5 moltiplicatori fisici. Le 11 iterazioni complessive vengono così ripartite in macro-blocchi eseguiti in time-multiplexing su una struttura hardware parzialmente ridondante, cercando un punto di ottimo intermedio nello spazio dei trade-off area-performance.

#### 4.4.6. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione `09_loop-fission-unroll-pipeline` mira a fondere i vantaggi del partizionamento hardware con la sovrapposizione temporale delle fasi di calcolo. Ci aspettiamo che l'assenza di conflitti sulle porte di memoria permetta alla pipeline del ciclo aritmetico di operare a pieno regime, abbattendo in modo drastico la latenza complessiva in cicli di clock

rispetto alla variante puramente spaziale della soluzione 07, pur mantenendo un impatto moderato sull'allocazione dei blocchi aritmetici avanzati.

## Timing

| Metric                       | 07 — Array Partitioning | 09 — Unroll + Pipeline (MAC) | $\Delta$  |
|------------------------------|-------------------------|------------------------------|-----------|
| Target clock                 | 10.000 ns               | 10.000 ns                    | —         |
| Estimated clock / $F_{\max}$ | 2.686 ns                | 4.226 ns                     | ▲ 57.33 % |
| Max achievable $F_{\max}$    | <b>372.300 MHz</b>      | <b>236.630 MHz</b>           | ▼ 36.44 % |
| Clock uncertainty            | 2.7 ns                  | 2.7 ns                       | —         |

## Latency and Throughput

| Metric                              | 07 — Array Partitioning                   | 09 — Unroll + Pipeline (MAC)           | $\Delta$  |
|-------------------------------------|-------------------------------------------|----------------------------------------|-----------|
| Latency max @ target clock          | 56cyc → 560.0 ns<br>(0.5600 $\mu$ s)      | 16cyc → 160.0 ns<br>(0.1600 $\mu$ s)   | ▼ 71.43 % |
| Latency max @ estimated clock       | 56cyc → 150.4 ns<br>(0.1504 $\mu$ s)      | 16cyc → 67.6 ns<br>(0.0676 $\mu$ s)    | ▼ 55.05 % |
| CoSim total exec. @ target clock    | 1709cyc → 17090.0 ns<br>(17.0900 $\mu$ s) | 509cyc → 5090.0 ns<br>(5.0900 $\mu$ s) | ▼ 70.22 % |
| CoSim total exec. @ estimated clock | 1709cyc → 4590.4 ns<br>(4.5904 $\mu$ s)   | 509cyc → 2151.0 ns<br>(2.1510 $\mu$ s) | ▼ 53.14 % |

## Resource Utilisation

| Metric                                                      | 07 — Array Partitioning | 09 — Unroll + Pipeline (MAC) | $\Delta$ |
|-------------------------------------------------------------|-------------------------|------------------------------|----------|
| LUT                                                         | 120 (0.23 %)            | 128 (0.24 %)                 | ▲ 6.67 % |
| FF                                                          | 310 (0.29 %)            | 338 (0.32 %)                 | ▲ 9.03 % |
| BRAM                                                        | 0 (0.00 %)              | 0 (0.00 %)                   | —        |
| DSP                                                         | 1 (0.45 %)              | 1 (0.45 %)                   | = 0.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                         |                              | ▲ 4.25 % |

## Power and Energy

| Metric | 07 — Array Partitioning | 09 — Unroll + Pipeline (MAC) | $\Delta$ |
|--------|-------------------------|------------------------------|----------|
|--------|-------------------------|------------------------------|----------|



|                                                  |                          |                          |           |
|--------------------------------------------------|--------------------------|--------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                  | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.002 W                  | 0.003 W                  | ▲ 50.00 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.105 W</b>           | <b>0.105 W</b>           | = 0.00 %  |
| Energy/op $E_{\text{op}}$                        | $1.579 \times 10^{-8}$ J | $7.100 \times 10^{-9}$ J | ▼ 55.05 % |
| Total energy $E_{\text{total}}$                  | $4.820 \times 10^{-7}$ J | $2.259 \times 10^{-7}$ J | ▼ 53.14 % |

## Figures of Merit

| Metric                     | 07 — Array Partitioning | 09 — Unroll + Pipeline (MAC) | $\Delta$  |
|----------------------------|-------------------------|------------------------------|-----------|
| Area-Delay Product (ADP)   | 44.5955                 | 21.7857                      | ▼ 51.15 % |
| Energy-Delay Product (EDP) | $2.2125 \times 10^{-3}$ | $4.8583 \times 10^{-4}$      | ▼ 78.04 % |

### Analisi delle metriche

L'analisi dei report evidenzia un incremento prestazionale straordinario sul fronte della latenza computazionale. Grazie alla convergenza delle direttive, la latenza per l'elaborazione di un singolo campione crolla da 56 a soli 16 **colpi di clock**, registrando una contrazione netta del 71.4%. Specularmente, il numero di *cicli totali* in co-simulazione sperimenta un abbattimento del 70.2%, scendendo da 1709 a 509. Questo incremento di throughput si paga con un innalzamento del periodo di clock stimato, che passa da 2.686 ns a 4.226 ns (un aumento del 57.3% del cammino critico) dovuto all'introduzione della logica di staging della pipeline. Tuttavia, il bilancio temporale assoluto rimane ampiamente positivo: il **Delay** reale crolla da 150.42 ns a soli 67.62 ns, segnando una velocizzazione netta del 55%. Sotto il profilo delle risorse hardware, l'architettura conserva una compattezza esemplare: le LUT combinatorie salgono in modo marginale da 120 a 128 (incremento del 6.7%) e i Flip-Flop passano da 310 a 338 (un aumento del 9%) per l'inserimento dei registri di pipeline. L'impiego dei blocchi **DSP** rimane ancorato al minimo di 1, confermando l'altissimo livello di riuso temporale dell'hardware.

### Analisi preliminare

Il confronto con la soluzione **10\_total-unroll** esamina il passaggio radicale dal parallelismo temporale (pipeline) al parallelismo spaziale puro (unrolling completo). L'aspettativa a livello hardware risiede nella totale eliminazione della natura iterativa della **FSM** di controllo aritmetica, proiettando la latenza del ciclo verso il minimo assoluto a spese di una massiccia allocazione di moltiplicatori fisici sul silicio.

## Timing

| Metric                       | 09 — Unroll + Pipeline (MAC) | 10 — Total Unroll (MAC) | $\Delta$  |
|------------------------------|------------------------------|-------------------------|-----------|
| Target clock                 | 10.000 ns                    | 10.000 ns               | —         |
| Estimated clock / $F_{\max}$ | 4.226 ns                     | 5.293 ns                | ▲ 25.25 % |
| Max achievable $F_{\max}$    | <b>236.630 MHz</b>           | <b>188.910 MHz</b>      | ▼ 20.17 % |
| Clock uncertainty            | 2.7 ns                       | 2.7 ns                  | —         |

### Latency and Throughput

| Metric                              | 09 — Unroll + Pipeline (MAC)           | 10 — Total Unroll (MAC)                | $\Delta$  |
|-------------------------------------|----------------------------------------|----------------------------------------|-----------|
| Latency max @ target clock          | 16cyc → 160.0 ns<br>(0.1600 $\mu$ s)   | 4cyc → 40.0 ns<br>(0.0400 $\mu$ s)     | ▼ 75.00 % |
| Latency max @ estimated clock       | 16cyc → 67.6 ns<br>(0.0676 $\mu$ s)    | 4cyc → 21.2 ns<br>(0.0212 $\mu$ s)     | ▼ 68.69 % |
| CoSim total exec. @ target clock    | 509cyc → 5090.0 ns<br>(5.0900 $\mu$ s) | 149cyc → 1490.0 ns<br>(1.4900 $\mu$ s) | ▼ 70.73 % |
| CoSim total exec. @ estimated clock | 509cyc → 2151.0 ns<br>(2.1510 $\mu$ s) | 149cyc → 788.7 ns<br>(0.7887 $\mu$ s)  | ▼ 63.34 % |

### Resource Utilisation

| Metric                                                      | 09 — Unroll + Pipeline (MAC) | 10 — Total Unroll (MAC) | $\Delta$   |
|-------------------------------------------------------------|------------------------------|-------------------------|------------|
| LUT                                                         | 128 (0.24 %)                 | 215 (0.40 %)            | ▲ 67.97 %  |
| FF                                                          | 338 (0.32 %)                 | 149 (0.14 %)            | ▼ 55.92 %  |
| BRAM                                                        | 0 (0.00 %)                   | 0 (0.00 %)              | —          |
| DSP                                                         | 1 (0.45 %)                   | 7 (3.18 %)              | ▲ 600.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                              |                         | ▲ 267.89 % |

### Power and Energy

| Metric                                           | 09 — Unroll + Pipeline (MAC) | 10 — Total Unroll (MAC)  | $\Delta$  |
|--------------------------------------------------|------------------------------|--------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                      | 0.103 W                  | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.003 W                      | 0.001 W                  | ▼ 66.67 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.105 W</b>               | <b>0.103 W</b>           | ▼ 1.90 %  |
| Energy/op $E_{\text{op}}$                        | $7.100 \times 10^{-9}$ J     | $2.181 \times 10^{-9}$ J | ▼ 69.28 % |

|                                 |                                  |                                  |                  |
|---------------------------------|----------------------------------|----------------------------------|------------------|
| Total energy $E_{\text{total}}$ | $2.259 \times 10^{-7} \text{ J}$ | $8.123 \times 10^{-8} \text{ J}$ | ▼ <b>64.03 %</b> |
|---------------------------------|----------------------------------|----------------------------------|------------------|

## Figures of Merit

| Metric                     |  | 09 — Unroll + Pipeline (MAC) | 10 — Total Unroll (MAC) | $\Delta$         |
|----------------------------|--|------------------------------|-------------------------|------------------|
| Area-Delay Product (ADP)   |  | 21.7857                      | 29.3854                 | ▲ <b>34.88 %</b> |
| Energy-Delay Product (EDP) |  | $4.8583 \times 10^{-4}$      | $6.4064 \times 10^{-5}$ | ▼ <b>86.81 %</b> |

### Analisi delle metriche

Le metriche documentano un salto evolutivo dirompente sul piano temporale. Lo srotolamento completo del loop abbatte la latenza algoritmica da 16 a soli **4 colpi di clock** (riduzione del 75%) e riduce i *cicli totali* d'esecuzione del 70.7%, facendoli precipitare al minimo storico di 149 cicli. Nonostante la complessità della rete combinatoria provochi un incremento del periodo di clock stimato, che sale da 4.226 ns a 5.293 ns (un aumento del 25.2%), l'effettivo **Delay** operativo reale crolla da 67.62 ns a soli 21.17 ns, conquistando un guadagno netto del 68.7%. Questa accelerazione estrema impone un pesante dazio in termini di area e trade-off fisici. L'istanza parallela dei componenti aritmetici fa esplodere l'impiego dei blocchi **DSP**, che balzano da 1 a 7 unità hardware attive. Parallelamente, le LUT combinatorie destinate a mappare la rete di interconnessione e l'albero di sommo salgono da 128 a 215 (un incremento del 68%). Si registra invece una contrazione contromano dei Flip-Flop, che scendono da 338 a 149 (riduzione del 55.9%) come diretta conseguenza dello smantellamento dei registri di sfasamento interni alla pipeline della soluzione precedente.

## Timing

| Metric                             | 10 — Total Unroll (MAC) | 11 — Partial Unroll (factor 5) | $\Delta$         |
|------------------------------------|-------------------------|--------------------------------|------------------|
| Target clock                       | 10.000 ns               | 10.000 ns                      | —                |
| Estimated clock / $F_{\text{max}}$ | 5.293 ns                | 6.859 ns                       | ▲ <b>29.59 %</b> |
| Max achievable $F_{\text{max}}$    | <b>188.910 MHz</b>      | <b>145.780 MHz</b>             | ▼ <b>22.83 %</b> |
| Clock uncertainty                  | 2.7 ns                  | 2.7 ns                         | —                |

## Latency and Throughput

| Metric | 10 — Total Unroll (MAC) | 11 — Partial Unroll (factor 5) | $\Delta$ |
|--------|-------------------------|--------------------------------|----------|
|--------|-------------------------|--------------------------------|----------|

|                                     |                                                    |                                                    |                   |
|-------------------------------------|----------------------------------------------------|----------------------------------------------------|-------------------|
| Latency max @ target clock          | 4cyc $\rightarrow$ 40.0 ns<br>(0.0400 $\mu$ s)     | 25cyc $\rightarrow$ 250.0 ns<br>(0.2500 $\mu$ s)   | <b>▲ 525.00 %</b> |
| Latency max @ estimated clock       | 4cyc $\rightarrow$ 21.2 ns<br>(0.0212 $\mu$ s)     | 25cyc $\rightarrow$ 171.5 ns<br>(0.1715 $\mu$ s)   | <b>▲ 709.91 %</b> |
| CoSim total exec. @ target clock    | 149cyc $\rightarrow$ 1490.0 ns<br>(1.4900 $\mu$ s) | 779cyc $\rightarrow$ 7790.0 ns<br>(7.7900 $\mu$ s) | <b>▲ 422.82 %</b> |
| CoSim total exec. @ estimated clock | 149cyc $\rightarrow$ 788.7 ns<br>(0.7887 $\mu$ s)  | 779cyc $\rightarrow$ 5343.2 ns<br>(5.3432 $\mu$ s) | <b>▲ 577.50 %</b> |

### Resource Utilisation

| Metric                                                      | 10 — Total Unroll<br>(MAC) | 11 — Partial Unroll<br>(factor 5) | $\Delta$          |
|-------------------------------------------------------------|----------------------------|-----------------------------------|-------------------|
| LUT                                                         | 215 (0.40 %)               | 387 (0.73 %)                      | <b>▲ 80.00 %</b>  |
| FF                                                          | 149 (0.14 %)               | 432 (0.41 %)                      | <b>▲ 189.93 %</b> |
| BRAM                                                        | 0 (0.00 %)                 | 0 (0.00 %)                        | —                 |
| DSP                                                         | 7 (3.18 %)                 | 5 (2.27 %)                        | <b>▼ 28.57 %</b>  |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                            |                                   | <b>▼ 8.58 %</b>   |

### Power and Energy

| Metric                                           | 10 — Total Unroll<br>(MAC) | 11 — Partial Unroll<br>(factor 5) | $\Delta$          |
|--------------------------------------------------|----------------------------|-----------------------------------|-------------------|
| Static power $P_{\text{static}}$                 | 0.103 W                    | 0.103 W                           | = 0.00 %          |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                    | 0.003 W                           | <b>▲ 200.00 %</b> |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>             | <b>0.105 W</b>                    | <b>▲ 1.94 %</b>   |
| Energy/op $E_{\text{op}}$                        | $2.181 \times 10^{-9}$ J   | $1.801 \times 10^{-8}$ J          | <b>▲ 725.54 %</b> |
| Total energy $E_{\text{total}}$                  | $8.123 \times 10^{-8}$ J   | $5.610 \times 10^{-7}$ J          | <b>▲ 590.65 %</b> |

### Figures of Merit

| Metric                     | 10 — Total Unroll<br>(MAC) | 11 — Partial Unroll<br>(factor 5) | $\Delta$           |
|----------------------------|----------------------------|-----------------------------------|--------------------|
| Area-Delay Product (ADP)   | 29.3854                    | 181.9988                          | <b>▲ 519.35 %</b>  |
| Energy-Delay Product (EDP) | $6.4064 \times 10^{-5}$    | $2.9977 \times 10^{-3}$           | <b>▲ 4579.19 %</b> |

## Analisi delle metriche

La valutazione della soluzione `11_partial-unroll` rispetto alla precedente configurazione piana evidenzia il fallimento della strategia di compromesso parziale in questo specifico contesto. Sotto il profilo temporale, il confinamento del datapath a soli 5 moltiplicatori degrada pesantemente la risposta dell'IP core: la latenza risale bruscamente da 4 a **25 colpi di clock** (un incremento superiore al 500%) e i *cicli totali* d'orologio balzano a 779. Il cammino critico combinatorio sperimenta un ulteriore e inatteso appesantimento, innalzando il periodo di clock stimato da 5.293 ns a 6.859 ns, a causa dell'introduzione della complessa logica di multiplexing dinamico necessaria per condividere i moltiplicatori tra i macro-blocchi del ciclo. Il **Delay** reale complessivo peggiora drasticamente, salendo a 171.47 ns. Il bilancio fallimentare di questa soluzione si riflette anche sull'efficienza d'area globale. Sebbene il numero di blocchi **DSP** sperimenti una contrazione marginale scendendo da 7 a 5 unità, le LUT combinatorie necessarie per governare l'indirizzamento e lo smistamento dei dati parziali subiscono un'impennata del 80%, passando da 215 a ben 387. Anche i Flip-Flop hardware registrano una crescita imponente, quasi triplicando da 149 a 432 per l'istanza dei registri di persistenza temporanea dei blocchi. L'architettura parziale risulta quindi nettamente inefficiente sotto ogni punto di vista, venendo completamente dominata dalla variante totale.

## Conclusioni

L'esplorazione del parallelismo sul datapath aritmetico ha decretato la netta superiorità delle soluzioni a scomposizione totale. La variante `10_total-unroll` stabilisce il record assoluto di velocità dell'intero progetto, riducendo il tempo reale d'esecuzione a soli 21.17 ns al costo di un'importante allocazione di risorse fisiche (7 DSP). Al contempo, la soluzione `09_loop-fission-unroll-pipeline` si posiziona come una frontiera di eccezionale efficienza intermedia, garantendo un ottimo **Delay** di 67.62 ns pur occupando un solo blocco DSP e un'area logica combinatoria estremamente contenuta (128 LUT).

Avendo mappato e compreso l'intera frontiera dei trade-off spaziali e temporali sulle architetture derivate dal frazionamento dei cicli, il piano di azione impone di effettuare un test di validazione incrociata per analizzare se sia possibile estrarre ulteriori margini di performance. Di conseguenza, i prossimi passi del workflow prevederanno di riprendere la soluzione sequenziale compatta ed efficiente nei tipi dati per **testare l'applicazione del Loop Pipelining direttamente sulla soluzione 05**. Questa sessione analitica parallela permetterà di verificare se la sovrapposizione temporale applicata a un datapath rollato a bit ridotti sia in grado di competere con la frontiera spaziale finora tracciata, ottimizzando i prodotti di trade-off d'area e delay.

### 4.4.7. Esplorazione Specialistica della Memoria: Concorrenza con «ap\_shift\_reg»

In questo capitolo il focus converge sull'applicazione dei paradigmi di parallelismo spaziale e temporale direttamente sopra la struttura di memoria di stato ottimizzata tramite la libreria specialistica Xilinx. Avendo consolidato l'efficacia d'area della classe template nella soluzione 04, l'obiettivo è analizzare se l'introduzione di direttive avanzate di srotolamento e di pipeline a livello di funzione possa estrarre ulteriori performance velocistiche dal datapath sequenziale

della libreria, confrontandone l'efficienza con le frontiere piane finora tracciate tramite il partizionamento manuale degli array.

#### 4.4.7.1. Soluzione 12: Loop Unrolling Totale su Struttura `ap_shift_reg`

##### 4.4.7.1.1. Descrizione dell'Implementazione

La soluzione `12_unroll` introduce il parallelismo spaziale completo all'interno della struttura logica della libreria dedicata. L'intervento software riprende il datapath a bit ridotti della soluzione 04 e applica la direttiva `#pragma HLS unroll` all'interno del corpo del ciclo aritmetico `mac_loop`:

```
...

// --- Shift REG ---
shift_reg.shift(x);

// Sequential MAC (Multiply-Accumulate) operations
mac_loop: for (int i = 0; i < N; i++) {
    #pragma HLS unroll /* OPTIMIZATION(unroll): unrolling the MAC operations

    // --- MAC ---
    acc += shift_reg.read(i) * c[i]; // Perform MAC on the i-th element
}

...
```

Codice 14: Srotolamento spaziale completo del ciclo aritmetico applicato sui canali di lettura della libreria `ap_shift_reg`.

A livello implementativo, questa configurazione costringe il sintetizzatore HLS a tentare la linearizzazione simultanea di tutte le 11 moltiplicazioni. Poiché la classe `ap_shift_reg` mappa nativamente i dati all'interno delle primitive *SRL* (*Shift Register LUT*), il compilatore deve distendere i nodi di lettura interni per alimentare in parallelo un albero di moltiplicatori fisici discreti, eliminando la natura iterativa della **FSM** di controllo del ciclo.

#### 4.4.7.2. Soluzione 13: Pipelining a Livello di Funzione con `ap_shift_reg`

##### 4.4.7.2.1. Descrizione dell'Implementazione

La soluzione `13_pipeline` persegue la massima ottimizzazione concorrente attraverso la sovrapposizione temporale globale delle fasi di esecuzione. L'intervento architetturale sposta la direttiva di ottimizzazione dal corpo dei singoli cicli ponendola direttamente all'inizio della funzione principale del core, inserendo il pragma `#pragma HLS pipeline II=1`:

```
...

void fir(out_data_t *y, in_data_t x) {
    #pragma HLS pipeline II=1 /* OPTIMIZATION(pipeline): executing FIR in pipeline

    // Constant coefficients array for the impulse response
    const coef_t c[N] = {53, 0, -91, 0, 313, 500, 313, 0, -91, 0, 53};
    static ap_shift_reg<in_data_t, N> shift_reg;

    ...
}
```

Codice 15: Applicazione del **Loop Pipelining** globale a livello di funzione (**Top-Level Pipelining**).

Questa trasformazione stravolge la schedulazione hardware dell'IP core: la direttiva impone al sintetizzatore di considerare l'intera funzione `fir` come un unico datapath pipelinato ad alto throughput. Di conseguenza, il compilatore srotola implicitamente ogni loop interno ed alloca registri di sfasamento hardware lungo l'intera catena logica (dall'ingresso del campione fino alla generazione dell'accumulo), predisponendo il circuito per accettare un nuovo input ad ogni colpo di clock ideale.

#### 4.4.8. Valutazione dei miglioramenti

##### Analisi preliminare

La soluzione `12_unroll` tenta di scavalcare la natura seriale della libreria specializzata distendendo nello spazio le risorse combinatorie. Ci aspettiamo che lo srotolamento completo abbatta drasticamente la latenza in cicli di clock rispetto alla configurazione sequenziale della soluzione `04`, accettando un incremento importante dell'area hardware dovuto all'istanza simultanea dei moltiplicatori paralleli.

##### Timing

| Metric                       | 04 — Library<br>ap_shift_reg | 12 — Loop unroll<br>(on v04) | $\Delta$   |
|------------------------------|------------------------------|------------------------------|------------|
| Target clock                 | 10.000 ns                    | 10.000 ns                    | —          |
| Estimated clock / $F_{\max}$ | 2.686 ns                     | 6.486 ns                     | ▲ 141.47 % |
| Max achievable $F_{\max}$    | <b>372.300 MHz</b>           | <b>154.190 MHz</b>           | ▼ 58.58 %  |
| Clock uncertainty            | 2.7 ns                       | 2.7 ns                       | —          |

##### Latency and Throughput

| Metric                              | 04 — Library<br>ap_shift_reg              | 12 — Loop unroll<br>(on v04)           | $\Delta$  |
|-------------------------------------|-------------------------------------------|----------------------------------------|-----------|
| Latency max @ target clock          | 56cyc → 560.0 ns<br>(0.5600 $\mu$ s)      | 10cyc → 100.0 ns<br>(0.1000 $\mu$ s)   | ▼ 82.14 % |
| Latency max @ estimated clock       | 56cyc → 150.4 ns<br>(0.1504 $\mu$ s)      | 10cyc → 64.9 ns<br>(0.0649 $\mu$ s)    | ▼ 56.88 % |
| CoSim total exec. @ target clock    | 1709cyc → 17090.0 ns<br>(17.0900 $\mu$ s) | 329cyc → 3290.0 ns<br>(3.2900 $\mu$ s) | ▼ 80.75 % |
| CoSim total exec. @ estimated clock | 1709cyc → 4590.4 ns<br>(4.5904 $\mu$ s)   | 329cyc → 2133.9 ns<br>(2.1339 $\mu$ s) | ▼ 53.51 % |

##### Resource Utilisation

| Metric | 04 — Library<br>ap_shift_reg | 12 — Loop unroll<br>(on v04) | $\Delta$ |
|--------|------------------------------|------------------------------|----------|
|--------|------------------------------|------------------------------|----------|

|                                                             |              |              |                   |
|-------------------------------------------------------------|--------------|--------------|-------------------|
| LUT                                                         | 107 (0.20 %) | 304 (0.57 %) | ▲ <b>184.11 %</b> |
| FF                                                          | 114 (0.11 %) | 166 (0.16 %) | ▲ <b>45.61 %</b>  |
| BRAM                                                        | 0 (0.00 %)   | 0 (0.00 %)   | —                 |
| DSP                                                         | 1 (0.45 %)   | 7 (3.18 %)   | ▲ <b>600.00 %</b> |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |              |              | ▲ <b>412.49 %</b> |

## Power and Energy

| Metric                                           | 04 — Library<br>ap_shift_reg | 12 — Loop unroll<br>(on v04) | $\Delta$         |
|--------------------------------------------------|------------------------------|------------------------------|------------------|
| Static power $P_{\text{static}}$                 | 0.103 W                      | 0.103 W                      | = 0.00 %         |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                      | 0.001 W                      | = 0.00 %         |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>               | <b>0.103 W</b>               | = 0.00 %         |
| Energy/op $E_{\text{op}}$                        | $1.549 \times 10^{-8}$ J     | $6.681 \times 10^{-9}$ J     | ▼ <b>56.88 %</b> |
| Total energy $E_{\text{total}}$                  | $4.728 \times 10^{-7}$ J     | $2.198 \times 10^{-7}$ J     | ▼ <b>53.51 %</b> |

## Figures of Merit

| Metric                     | 04 — Library<br>ap_shift_reg | 12 — Loop unroll<br>(on v04) | $\Delta$          |
|----------------------------|------------------------------|------------------------------|-------------------|
| Area-Delay Product (ADP)   | 35.0154                      | 83.4203                      | ▲ <b>138.24 %</b> |
| Energy-Delay Product (EDP) | $2.1704 \times 10^{-3}$      | $4.6901 \times 10^{-4}$      | ▼ <b>78.39 %</b>  |

### Analisi delle metriche

I dati estratti confermano un netto incremento del throughput algoritmico a scapito delle risorse fisiche. Sul fronte temporale, la latenza di calcolo sperimenta un crollo verticale, precipitando da 56 a soli 10 **colpi di clock** (un risparmio netto del 82.1%) con i *cicli totali* di esecuzione in co-simulazione che scendono a 329. Nonostante la complessità della rete combinatoria causi un innalzamento del periodo di clock stimato da 2.686 ns a 6.486 ns (un appesantimento del 141.5% del cammino critico), il tempo reale assoluto complessivo migliora sensibilmente, scendendo da 150.42 ns a 64.86 ns (riduzione del 56.9% del **Delay** operativo). Questo guadagno temporale esige un pesante dazio in termini di silicio: l'istanza parallela del datapath fa balzare i blocchi **DSP** da 1 a 7 unità hardware attive. Le LUT combinatorie destinate a governare la rete piana e l'albero di sommo subiscono un'impennata imponente, salendo da 107 a 304 (un incremento del 184.1%), mentre i Flip-Flop hardware registrano una crescita più contenuta, passando da 114 a 166 (+45.6%).



### Analisi preliminare

La soluzione 13\_pipeline applica il parallelismo temporale massimo sul datapath specializzato. L'aspettativa hardware risiede nella completa sovrapposizione delle fasi di elaborazione di campioni consecutivi, puntando a contrarre drasticamente l'Initiation Interval (II) globale rispetto alla variante sequenziale di riferimento della soluzione 04.

### Timing

| Metric                       | 04 — Library<br>ap_shift_reg | 13 — Function Pi-<br>pipeline (on v04) | $\Delta$   |
|------------------------------|------------------------------|----------------------------------------|------------|
| Target clock                 | 10.000 ns                    | 10.000 ns                              | —          |
| Estimated clock / $F_{\max}$ | 2.686 ns                     | 6.486 ns                               | ▲ 141.47 % |
| Max achievable $F_{\max}$    | <b>372.300 MHz</b>           | <b>154.190 MHz</b>                     | ▼ 58.58 %  |
| Clock uncertainty            | 2.7 ns                       | 2.7 ns                                 | —          |

### Latency and Throughput

| Metric                              | 04 — Library<br>ap_shift_reg              | 13 — Function Pi-<br>pipeline (on v04) | $\Delta$  |
|-------------------------------------|-------------------------------------------|----------------------------------------|-----------|
| Latency max @ target clock          | 56cyc → 560.0 ns<br>(0.5600 $\mu$ s)      | 10cyc → 100.0 ns<br>(0.1000 $\mu$ s)   | ▼ 82.14 % |
| Latency max @ estimated clock       | 56cyc → 150.4 ns<br>(0.1504 $\mu$ s)      | 10cyc → 64.9 ns<br>(0.0649 $\mu$ s)    | ▼ 56.88 % |
| CoSim total exec. @ target clock    | 1709cyc → 17090.0 ns<br>(17.0900 $\mu$ s) | 242cyc → 2420.0 ns<br>(2.4200 $\mu$ s) | ▼ 85.84 % |
| CoSim total exec. @ estimated clock | 1709cyc → 4590.4 ns<br>(4.5904 $\mu$ s)   | 242cyc → 1569.6 ns<br>(1.5696 $\mu$ s) | ▼ 65.81 % |

### Resource Utilisation

| Metric                                                      | 04 — Library<br>ap_shift_reg | 13 — Function Pi-<br>pipeline (on v04) | $\Delta$   |
|-------------------------------------------------------------|------------------------------|----------------------------------------|------------|
| LUT                                                         | 107 (0.20 %)                 | 312 (0.59 %)                           | ▲ 191.59 % |
| FF                                                          | 114 (0.11 %)                 | 215 (0.20 %)                           | ▲ 88.60 %  |
| BRAM                                                        | 0 (0.00 %)                   | 0 (0.00 %)                             | —          |
| DSP                                                         | 1 (0.45 %)                   | 7 (3.18 %)                             | ▲ 600.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                              |                                        | ▲ 420.50 % |

### Power and Energy

| Metric                                           | 04 — Library<br>ap_shift_reg | 13 — Function Pi-<br>pipeline (on v04) | $\Delta$   |
|--------------------------------------------------|------------------------------|----------------------------------------|------------|
| Static power $P_{\text{static}}$                 | 0.103 W                      | 0.103 W                                | = 0.00 %   |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                      | 0.002 W                                | ▲ 100.00 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>               | <b>0.104 W</b>                         | ▲ 0.97 %   |
| Energy/op $E_{\text{op}}$                        | $1.549 \times 10^{-8}$ J     | $5.396 \times 10^{-9}$ J               | ▼ 65.17 %  |
| Total energy $E_{\text{total}}$                  | $4.728 \times 10^{-7}$ J     | $1.632 \times 10^{-7}$ J               | ▼ 65.47 %  |

## Figures of Merit

| Metric                     | 04 — Library<br>ap_shift_reg | 13 — Function Pi-<br>pipeline (on v04) | $\Delta$  |
|----------------------------|------------------------------|----------------------------------------|-----------|
| Area-Delay Product (ADP)   | 35.0154                      | 62.3199                                | ▲ 77.98 % |
| Energy-Delay Product (EDP) | $2.1704 \times 10^{-3}$      | $2.5622 \times 10^{-4}$                | ▼ 88.19 % |

### Analisi delle metriche

Il confronto metrico convalida l'efficacia della pipeline di funzione nel superamento dei limiti seriali. Sotto il profilo delle performance d'orologio, la latenza del core si riduce da 56 a 10 *cicli* (riduzione del 82.1%) e l'Initiation Interval (II) sperimenta un abbattimento formidabile, crollando da 57 a soli 8 *cicli* di clock (un incremento di throughput del 85.9%). Il numero di *cicli totali* di simulazione si attesta al record parziale di 242. Il cammino critico si stabilizza sul valore di 6.486 ns, portando il tempo reale d'esecuzione della funzione a 64.86 ns. Sul piano d'area hardware, l'imposizione della pipeline a livello di **Top-Level** forza il compilatore a un'allocazione risorse speculare all'unrolling spaziale: vengono impiegati 7 blocchi **DSP**, mentre le LUT combinatorie salgono a 312 (un aumento del 191.6%) ed i Flip-Flop hardware si espandono a 215 (incremento del 88.6%) per l'istanza delle indispensabili barriere di registri di pipeline lungo le interconnessioni.

## Timing

| Metric                             | 10 — Total Unroll<br>(MAC) | 12 — Loop unroll<br>(on v04) | $\Delta$  |
|------------------------------------|----------------------------|------------------------------|-----------|
| Target clock                       | 10.000 ns                  | 10.000 ns                    | —         |
| Estimated clock / $F_{\text{max}}$ | 5.293 ns                   | 6.486 ns                     | ▲ 22.54 % |
| Max achievable $F_{\text{max}}$    | <b>188.910 MHz</b>         | <b>154.190 MHz</b>           | ▼ 18.38 % |
| Clock uncertainty                  | 2.7 ns                     | 2.7 ns                       | —         |

## Latency and Throughput

| Metric                              | 10 — Total Unroll (MAC)                            | 12 — Loop unroll (on v04)                          | $\Delta$                  |
|-------------------------------------|----------------------------------------------------|----------------------------------------------------|---------------------------|
| Latency max @ target clock          | 4cyc $\rightarrow$ 40.0 ns<br>(0.0400 $\mu$ s)     | 10cyc $\rightarrow$ 100.0 ns<br>(0.1000 $\mu$ s)   | $\blacktriangle$ 150.00 % |
| Latency max @ estimated clock       | 4cyc $\rightarrow$ 21.2 ns<br>(0.0212 $\mu$ s)     | 10cyc $\rightarrow$ 64.9 ns<br>(0.0649 $\mu$ s)    | $\blacktriangle$ 206.35 % |
| CoSim total exec. @ target clock    | 149cyc $\rightarrow$ 1490.0 ns<br>(1.4900 $\mu$ s) | 329cyc $\rightarrow$ 3290.0 ns<br>(3.2900 $\mu$ s) | $\blacktriangle$ 120.81 % |
| CoSim total exec. @ estimated clock | 149cyc $\rightarrow$ 788.7 ns<br>(0.7887 $\mu$ s)  | 329cyc $\rightarrow$ 2133.9 ns<br>(2.1339 $\mu$ s) | $\blacktriangle$ 170.57 % |

### Resource Utilisation

| Metric                                                      | 10 — Total Unroll (MAC) | 12 — Loop unroll (on v04) | $\Delta$                 |
|-------------------------------------------------------------|-------------------------|---------------------------|--------------------------|
| LUT                                                         | 215 (0.40 %)            | 304 (0.57 %)              | $\blacktriangle$ 41.40 % |
| FF                                                          | 149 (0.14 %)            | 166 (0.16 %)              | $\blacktriangle$ 11.41 % |
| BRAM                                                        | 0 (0.00 %)              | 0 (0.00 %)                | —                        |
| DSP                                                         | 7 (3.18 %)              | 7 (3.18 %)                | = 0.00 %                 |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                         |                           | $\blacktriangle$ 4.92 %  |

### Power and Energy

| Metric                                           | 10 — Total Unroll (MAC)  | 12 — Loop unroll (on v04) | $\Delta$                  |
|--------------------------------------------------|--------------------------|---------------------------|---------------------------|
| Static power $P_{\text{static}}$                 | 0.103 W                  | 0.103 W                   | = 0.00 %                  |
| Dynamic power $P_{\text{dynamic}}$               | 0.001 W                  | 0.001 W                   | = 0.00 %                  |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.103 W</b>           | <b>0.103 W</b>            | = 0.00 %                  |
| Energy/op $E_{\text{op}}$                        | $2.181 \times 10^{-9}$ J | $6.681 \times 10^{-9}$ J  | $\blacktriangle$ 206.33 % |
| Total energy $E_{\text{total}}$                  | $8.123 \times 10^{-8}$ J | $2.198 \times 10^{-7}$ J  | $\blacktriangle$ 170.57 % |

### Figures of Merit

| Metric                     | 10 — Total Unroll (MAC) | 12 — Loop unroll (on v04) | $\Delta$                  |
|----------------------------|-------------------------|---------------------------|---------------------------|
| Area-Delay Product (ADP)   | 29.3854                 | 83.4203                   | $\blacktriangle$ 183.88 % |
| Energy-Delay Product (EDP) | $6.4064 \times 10^{-5}$ | $4.6901 \times 10^{-4}$   | $\blacktriangle$ 632.09 % |

### Analisi delle metriche

Mettendo a confronto lo srotolamento combinato su libreria (**12\_unroll**) rispetto all'unrolling piallato su array completamente partizionato (**10 – Total Unroll**), si delinea la netta inefficienza della libreria nel dominio del parallelismo spaziale puro. Sul piano temporale, la soluzione 12 sperimenta un severo peggioramento delle metriche: la latenza algoritmica sale da 4 a 10 **colpi di clock** (un incremento del 150%) e il periodo di clock stimato si appesantisce da 5.293 ns a 6.486 ns (+22.5%) a causa dell'incompatibilità architetturale tra lo srotolamento combinatorio e la struttura sequenziale interna dei canali della macro-cella. Di conseguenza, l'effettivo **Delay** reale crolla in modo negativo, triplicando da 21.17 ns a 64.86 ns. Questo degrado prestazionale non è compensato da alcun risparmio d'area sul silicio. A parità di blocchi **DSP** impiegati (7 per entrambe le soluzioni), l'architettura basata su libreria richiede un volume di LUT combinatorie nettamente superiore, salendo da 215 a 304 (un incremento del 41.4%) dovuto alla complessa logica di indirizzamento dinamico interna alle primitive SRL, laddove la scomposizione fisica in registri discreti della soluzione 10 azzerava l'overhead combinatorio. I Flip-Flop hardware registrano un incremento marginale passando da 149 a 166 (+11.4%).

### Timing

| Metric                       | 09 — Unroll + Pipeline (MAC) | 13 — Function Pipeline (on v04) | $\Delta$         |
|------------------------------|------------------------------|---------------------------------|------------------|
| Target clock                 | 10.000 ns                    | 10.000 ns                       | —                |
| Estimated clock / $F_{\max}$ | 4.226 ns                     | 6.486 ns                        | ▲ <b>53.48 %</b> |
| Max achievable $F_{\max}$    | <b>236.630 MHz</b>           | <b>154.190 MHz</b>              | ▼ <b>34.84 %</b> |
| Clock uncertainty            | 2.7 ns                       | 2.7 ns                          | —                |

### Latency and Throughput

| Metric                              | 09 — Unroll + Pipeline (MAC)           | 13 — Function Pipeline (on v04)        | $\Delta$         |
|-------------------------------------|----------------------------------------|----------------------------------------|------------------|
| Latency max @ target clock          | 16cyc → 160.0 ns<br>(0.1600 $\mu$ s)   | 10cyc → 100.0 ns<br>(0.1000 $\mu$ s)   | ▼ <b>37.50 %</b> |
| Latency max @ estimated clock       | 16cyc → 67.6 ns<br>(0.0676 $\mu$ s)    | 10cyc → 64.9 ns<br>(0.0649 $\mu$ s)    | ▼ <b>4.08 %</b>  |
| CoSim total exec. @ target clock    | 509cyc → 5090.0 ns<br>(5.0900 $\mu$ s) | 242cyc → 2420.0 ns<br>(2.4200 $\mu$ s) | ▼ <b>52.46 %</b> |
| CoSim total exec. @ estimated clock | 509cyc → 2151.0 ns<br>(2.1510 $\mu$ s) | 242cyc → 1569.6 ns<br>(1.5696 $\mu$ s) | ▼ <b>27.03 %</b> |

### Resource Utilisation

| Metric                                                      | 09 — Unroll + Pipeline (MAC) | 13 — Function Pipeline (on v04) | $\Delta$   |
|-------------------------------------------------------------|------------------------------|---------------------------------|------------|
| LUT                                                         | 128 (0.24 %)                 | 312 (0.59 %)                    | ▲ 143.75 % |
| FF                                                          | 338 (0.32 %)                 | 215 (0.20 %)                    | ▼ 36.39 %  |
| BRAM                                                        | 0 (0.00 %)                   | 0 (0.00 %)                      | —          |
| DSP                                                         | 1 (0.45 %)                   | 7 (3.18 %)                      | ▲ 600.00 % |
| $A_{\text{norm}} = \Sigma (\text{used} / \text{available})$ |                              |                                 | ▲ 292.02 % |

## Power and Energy

| Metric                                           | 09 — Unroll + Pipeline (MAC) | 13 — Function Pipeline (on v04) | $\Delta$  |
|--------------------------------------------------|------------------------------|---------------------------------|-----------|
| Static power $P_{\text{static}}$                 | 0.103 W                      | 0.103 W                         | = 0.00 %  |
| Dynamic power $P_{\text{dynamic}}$               | 0.003 W                      | 0.002 W                         | ▼ 33.33 % |
| <b>Total power <math>P_{\text{total}}</math></b> | <b>0.105 W</b>               | <b>0.104 W</b>                  | ▼ 0.95 %  |
| Energy/op $E_{\text{op}}$                        | $7.100 \times 10^{-9}$ J     | $5.396 \times 10^{-9}$ J        | ▼ 24.00 % |
| Total energy $E_{\text{total}}$                  | $2.259 \times 10^{-7}$ J     | $1.632 \times 10^{-7}$ J        | ▼ 27.72 % |

## Figures of Merit

| Metric                     | 09 — Unroll + Pipeline (MAC) | 13 — Function Pipeline (on v04) | $\Delta$   |
|----------------------------|------------------------------|---------------------------------|------------|
| Area-Delay Product (ADP)   | 21.7857                      | 62.3199                         | ▲ 186.06 % |
| Energy-Delay Product (EDP) | $4.8583 \times 10^{-4}$      | $2.5622 \times 10^{-4}$         | ▼ 47.26 %  |

### Analisi delle metriche

Confrontando infine la pipeline di funzione (13\_pipeline) con la pipeline di ciclo fissionata su registri discreti (09 – Unroll + Pipeline), emerge un trade-off di eccezionale rilievo metodologico. Dal punto di vista della velocità pura, la pipeline a livello di top function della soluzione 13 si dimostra decisamente superiore: abbatte la latenza da 16 a 10 *cicli* (guadagno del 37.5%) e taglia a metà l’Initiation Interval (II) effettivo, che passa da 17 a soli 8 *cicli* di clock (miglioramento del 52.9%) riducendo i *cicli totali* da 509 a 242. Tuttavia, sotto il profilo dell’efficienza d’area e dello sfruttamento del silicio, la soluzione 13 risulta straordinariamente onerosa: l’impossibilità di condividere l’hardware nel tempo fa esplodere l’impiego dei blocchi **DSP**, che balzano da 1 a 7 unità fisiche attive. Le LUT combinatorie destinate al controllo e all’instradamento subiscono un’impennata del 143.7%, passando da 128 a 312. Si rileva di contro un eccellente risparmio sul fronte dei Flip-Flop combinatori,

che scendono da 338 a 215 (una contrazione del 36.4%) grazie alla capacità della libreria Xilinx di internalizzare i registri di sfasamento dentro le celle logiche delle SRL. Il periodo di clock stimato sperimenta un incremento da 4.226 ns a 6.486 ns.

## Conclusioni

L'esplorazione basata sulla libreria specialistica `ap_shift_reg` ha tracciato i confini fisici tra specializzazione d'area e parallelismo. L'architettura `13_pipeline` stabilisce il record assoluto di minor numero di cicli totali di esecuzione (242 cicli) e il minor volume di Flip-Flop tra le soluzioni ad alto throughput, confermando l'efficacia delle primitive SRL nel contenimento dell'area sequenziale. Tuttavia, il blocco interno della libreria impedisce al sintetizzatore di spingere il cammino critico combinatorio al di sotto dei 6.486 ns, lasciando la palma di massima velocità assoluta alla variante piana partizionata della soluzione 10 (21.17 ns).

Con questa sessione analitica si esaurisce la fase di esplorazione indipendente del parallelismo e della precisione dei tipi dati a livello di codice sorgente. Avendo mappato l'intera frontiera dei trade-off area-performance, il piano di azione impone di passare alla fase di ottimizzazione temporale avanzata a livello di clock. Di conseguenza, i prossimi passi fondamentali del workflow prevederanno di selezionare le soluzioni architetturali più valide stimate finora per **eseguire la tecnica di Operation Chaining avanzata**, modificando sistematicamente il vincolo temporale del clock per forzare il sintetizzatore HLS a fondere più operazioni logiche all'interno dello stesso ciclo d'orologio, ottimizzando i prodotti di merito d'area e delay.

## 4.5. Operation Chaining

### 4.5.1. Scelta delle soluzioni

In questa fase finale del flusso di progettazione, l'obiettivo si sposta dall'ottimizzazione del codice C/C++ all'esplorazione dei limiti fisici imposti dal sintetizzatore HLS tramite la tecnica dell'**Operation Chaining**. Modificando il vincolo del periodo di clock (esplorando scenari a 8.0 ns, 10.0 ns e 15.0 ns), si forza il compilatore ad aggregare o segmentare le operazioni combinatorie all'interno dei singoli cicli di clock.

Per condurre questo **Stress Test**, sono state selezionate tre architetture cardine che rappresentano i punti di ottimo nelle rispettive categorie esplorative:

- **Soluzione 03 — Bitwidth opt:** L'ottimo per compattezza e precisione dei tipi nel dominio puramente sequenziale.
- **Soluzione 07 — Array Partitioning:** La transizione ideale, caratterizzata da una memoria hardware sbloccata ma con controllo iterativo.
- **Soluzione 10 — Total Unroll (MAC):** Il vertice prestazionale in termini di parallelismo spaziale puro.

L'analisi si baserà sui dati esposti in due elaborazioni grafiche fondamentali: un **grafico a barre**, che evidenzia l'impatto dei constraint di clock sul **Delay** totale (tempo di esecuzione assoluto), e un **grafico scatter**, che mappa l'Area rispetto al Delay per tracciare la **Curva di Pareto** e identificare in modo visivo i trade-off delle configurazioni dominanti.

#### 4.5.2. Soluzione 03 — Bitwidth opt (on v01)

La soluzione 03 si affida a un'architettura rigorosamente sequenziale, in cui ogni colpo di clock gestisce un singolo aggiornamento dello shift register e una sola operazione aritmetica MAC. A causa di questo limite strutturale intrinseco, le operazioni delle iterazioni successive non possono essere concatenate nello stesso istante.

L'analisi dei grafici rivela che l'**Operation Chaining** è praticamente inefficace su questa configurazione. Il cammino critico combinatorio è estremamente breve (stimato a 3.132 ns) grazie ai tipi di dato ridotti. Di conseguenza, sia stringendo il vincolo a 8.0 ns sia rilassandolo a 15.0 ns, la schedulazione non subisce variazioni: la latenza algoritmica rimane rigidamente ancorata a 54 cicli. Nel grafico scatter, i punti relativi a questa soluzione risultano perfettamente sovrapposti, mantenendo un **Delay** costante di circa 169.12 ns e un'area logica fissa di 120 LUT e 77 Flip-Flop, confermando un'architettura stabile ma del tutto insensibile ai constraint temporali.

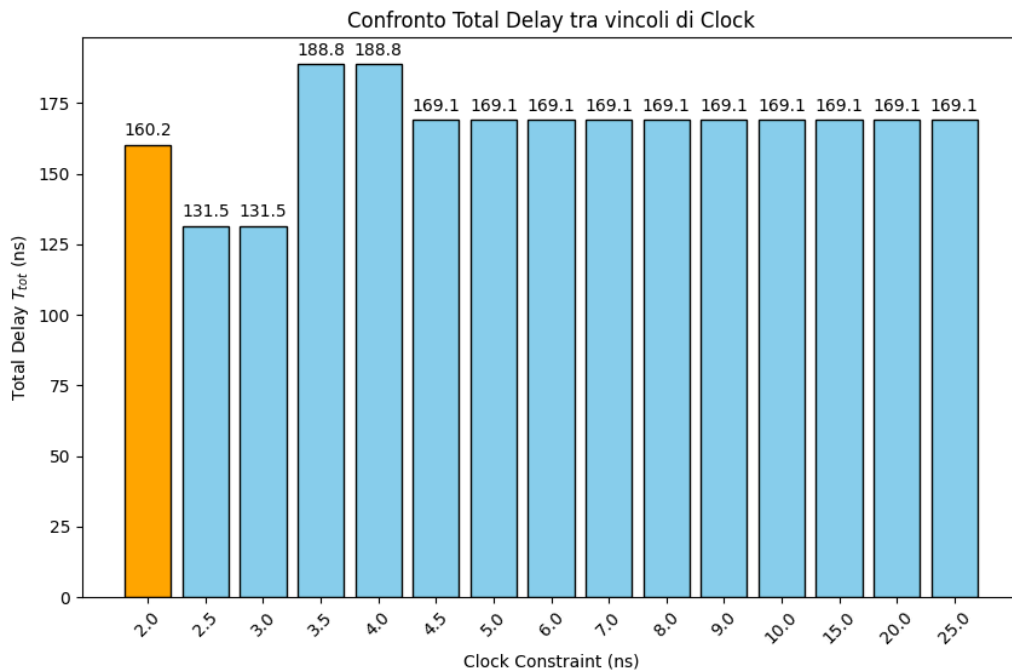


Figura 1: Analisi delle prestazioni totali.

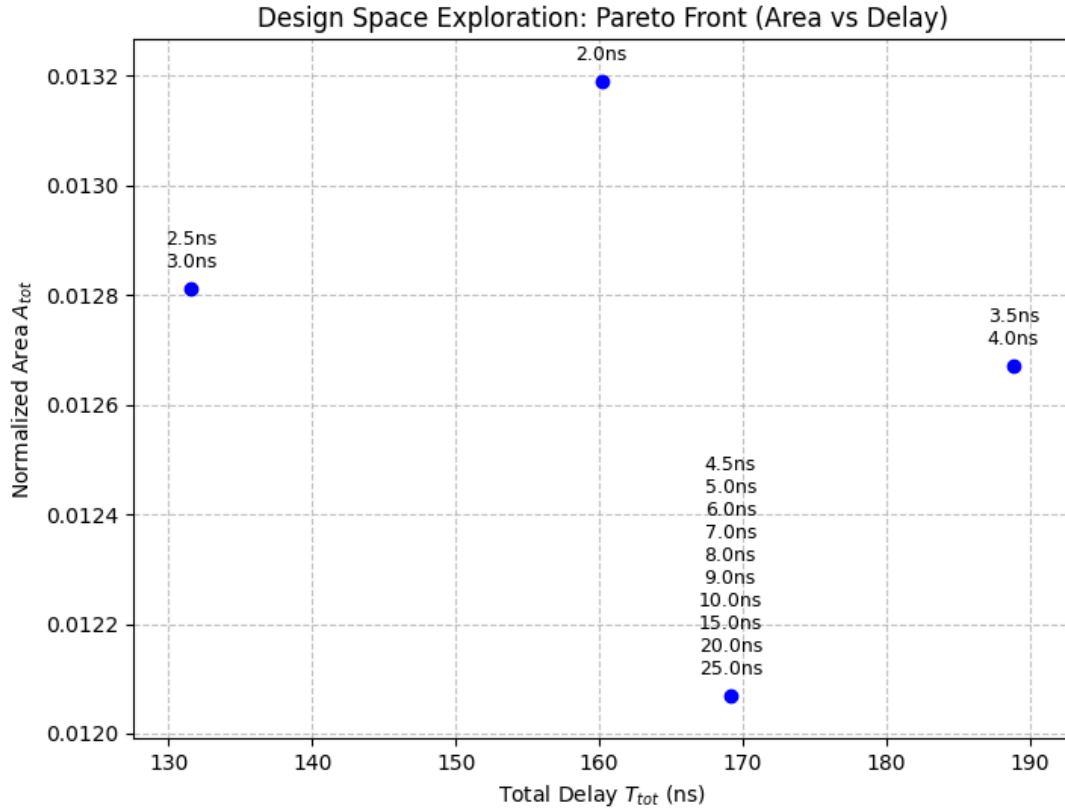


Figura 2: Analisi trade-off.

#### 4.5.3. Soluzione 07 — Array Partitioning

Sebbene la soluzione 07 introduca la completa frammentazione spaziale della memoria di stato in registri discreti, il ciclo di accumulo aritmetico rimane non srotolato. Questo fa sì che, esattamente come per la soluzione precedente, il flusso di calcolo rimanga dominato dal vincolo di dipendenza dei dati tra un'iterazione e l'altra.

Il periodo di clock stimato per questa architettura si assesta sul valore eccellente di 2.686 ns, il più rapido dell'intero workflow, garantito dalla totale assenza di conflitti sulle porte di memoria. Osservando il grafico a barre del ritardo totale, è evidente come variare il target di clock non induca il compilatore a modificare il numero di cicli (56 cicli), poiché non vi sono ulteriori operazioni seriali raggruppabili. Il **Delay** totale si mantiene inalterato a 150.41 ns. L'invarianza delle metriche colloca anche questa variante in una posizione statica sulla curva di Pareto, consolidando un profilo a basso consumo di LUT (120) ma con un impiego di Flip-Flop superiore rispetto alla soluzione 03 per via dei registri partizionati (310).



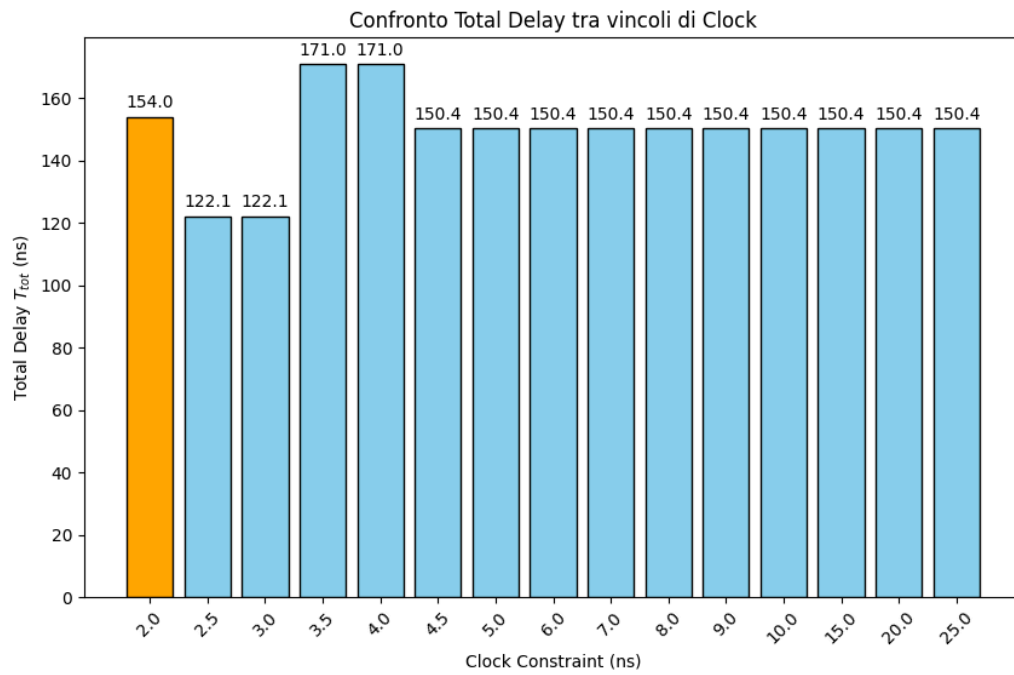


Figura 3: Analisi delle prestazioni totali.

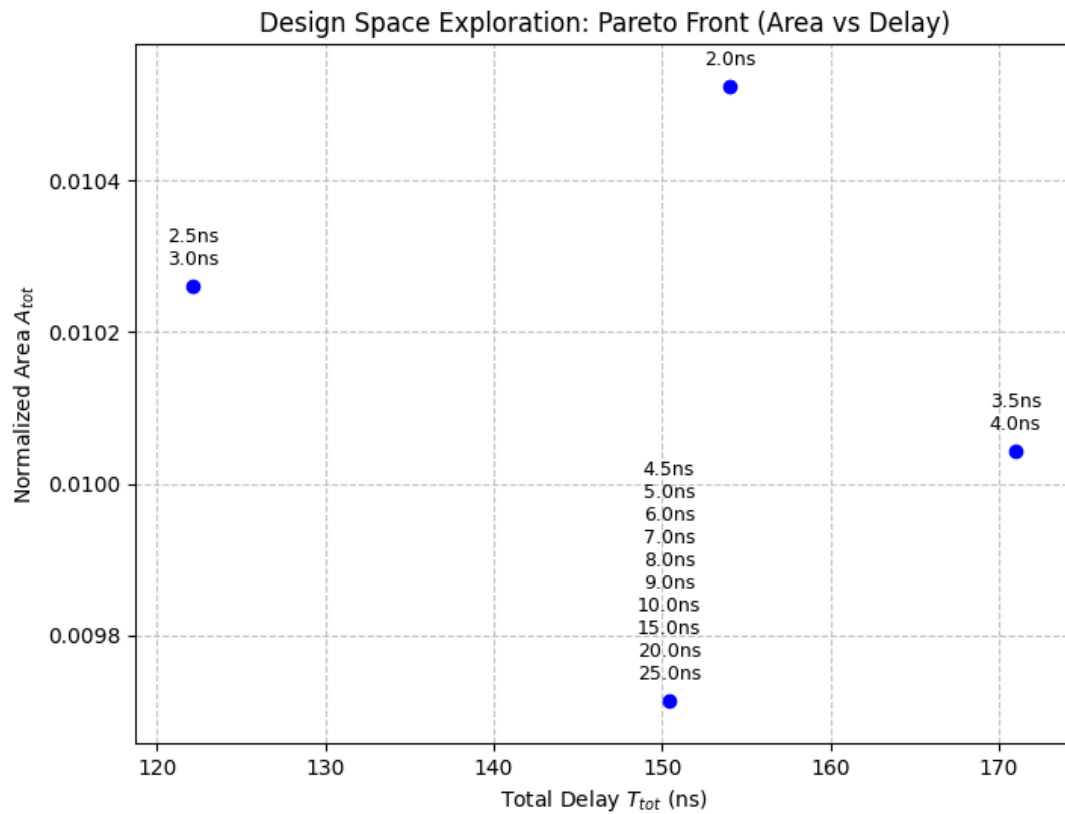


Figura 4: Analisi trade-off.

#### 4.5.4. Soluzione 10 — Total Unroll (MAC)

La soluzione 10 rappresenta lo scenario perfetto per osservare la potenza dell'**Operation Chaining**. Grazie allo srotolamento completo del loop, il compilatore HLS vede l'intero algoritmo come un'enorme rete combinatoria piana, composta dall'esecuzione simultanea di tutte le moltiplicazioni seguite da un vasto albero di sommatore, senza vincoli iterativi.

In questo caso, il grafico a barre mostra chiare variazioni del **Delay** totale al mutare del vincolo. Con constraint severi (come 10.0 ns o 8.0 ns), il tool è obbligato a «tagliare» il lungo cammino critico inserendo registri intermedi (pipelining combinatorio). Questo processo fissa il clock stimato intorno ai 5.293 ns e distribuisce il calcolo su 4 cicli (per un tempo totale di circa 21.17 ns). Al contrario, rilassando il constraint a 15.0 ns, il sintetizzatore sfrutta la finestra temporale più ampia per incatenare un numero maggiore di addizioni consecutive senza interporre Flip-Flop. Questa strategia contrae in modo aggressivo gli stati della **FSM**, riducendo la latenza in cicli ma provocando piccole fluttuazioni sia sull'area logica che sul tempo di esecuzione assoluto, chiaramente visibili nel grafico scatter. Sulla frontiera di Pareto, questa architettura si staglia nettamente come la configurazione dominante per le alte prestazioni, isolandosi all'estrema sinistra e illustrando i classici compromessi tra allocazione massima delle risorse (7 DSP, 215 LUT) e massimizzazione del throughput.

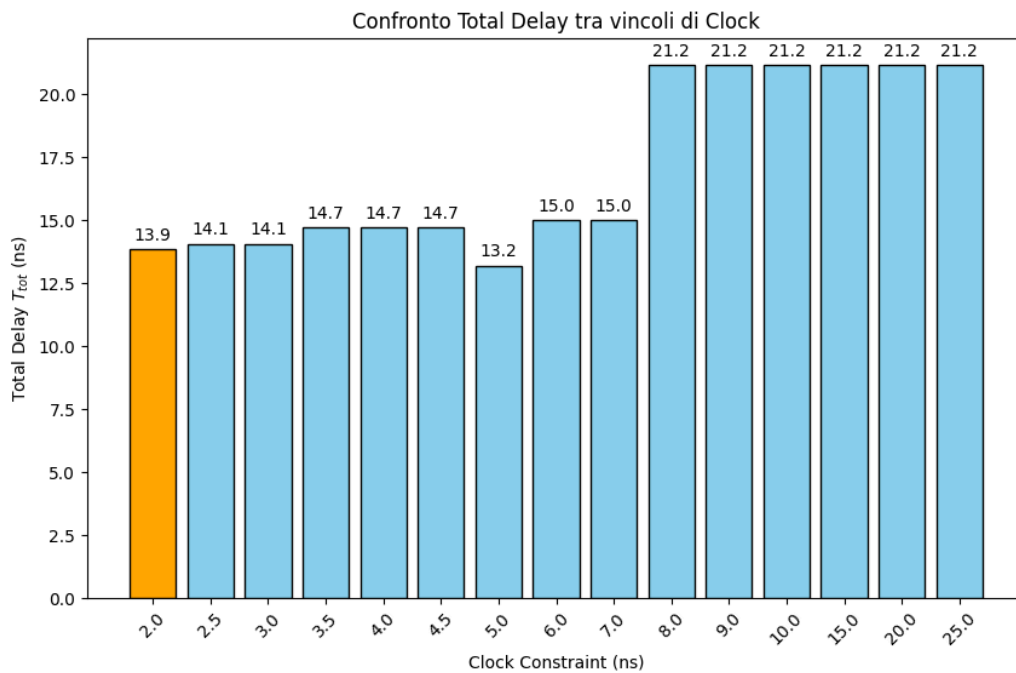


Figura 5: Analisi delle prestazioni totali.

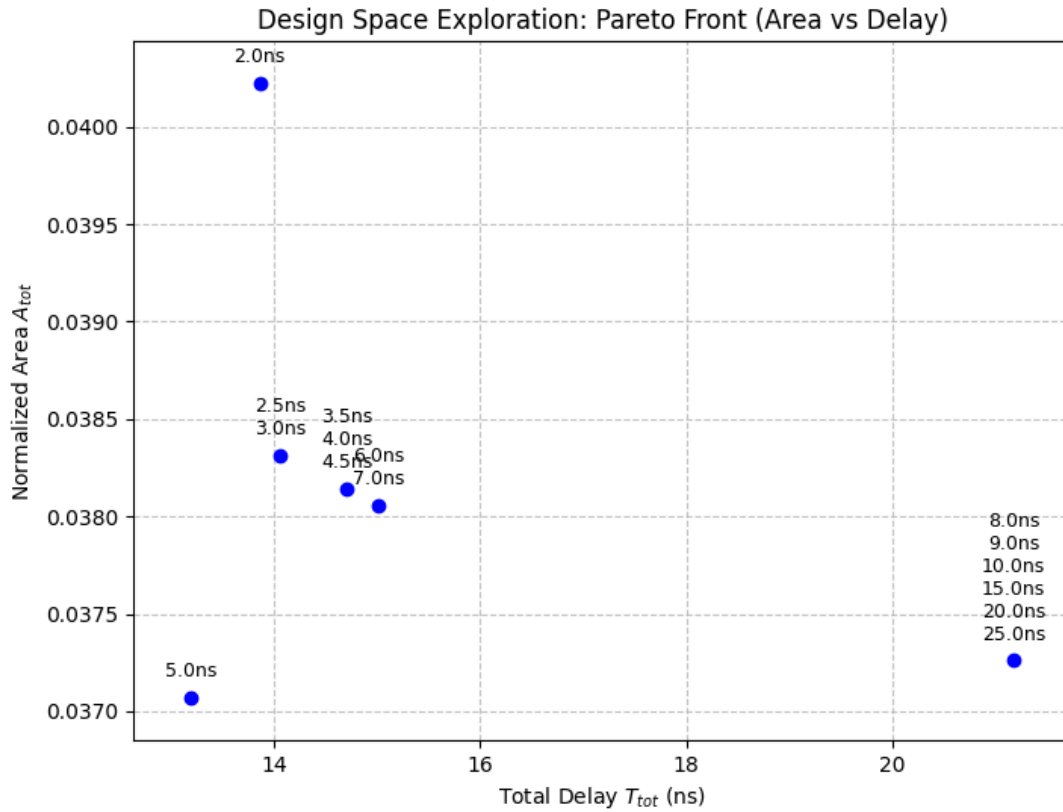


Figura 6: Analisi trade-off.

## 5. Sintesi dei Risultati e Conclusioni

### 5.1. Confronto Aggregato delle Soluzioni

L'esplorazione dello spazio del design (**Design Space Exploration**) ha permesso di tracciare in modo sistematico l'evoluzione dell'IP core FIR, partendo da una stesura software C/C++ *baseline* fino a giungere ad architetture hardware altamente specializzate. L'approccio incrementale ha dimostrato come la semplice traduzione di codice sequenziale in logica RTL generi circuiti inefficienti, rendendo indispensabile l'intervento del progettista tramite pragmi e refactoring strutturale.

L'analisi visiva aggregata, supportata dal grafico a dispersione (**Scatter Plot**), ha permesso di mappare ogni configurazione nello spazio **Area-Delay**. Questo strumento si è rivelato fondamentale per identificare la **Frontiera di Pareto**: l'insieme delle soluzioni dominanti per cui non è fisicamente possibile migliorare le prestazioni temporali senza incorrere in un proporzionale aumento dell'area occupata sul silicio. Si è osservata una chiara migrazione dei design da una zona ad alta latenza e bassa area (soluzioni sequenziali e ottimizzate nei bit) verso l'estremo a bassissima latenza e alta occupazione di DSP e registri (soluzioni completamente srotolate o pipelinate a livello di funzione).

### 5.2. Analisi delle Migliori Soluzioni

Dall'analisi della curva di Pareto e dei prodotti di trade-off (**ADP** ed **EDP**), è possibile decretare le architetture vincitrici in base ai diversi vincoli di progetto ipotizzabili:

- **Migliore per Area e Consumo (Low-Power / Edge Computing):** La **Soluzione 04** (Libreria `ap_shift_reg` con Bitwidth Optimization) e la **Soluzione 05** (Loop Fission + `ap_int`) rappresentano il vertice dell'efficienza spaziale. Riducendo i tipi di dato alla precisione strettamente necessaria (es. campioni a 12 *bit*, coefficienti a 10 *bit*) e sfruttando le primitive *SRL* per la memoria, queste varianti limitano l'uso dei blocchi **DSP** al minimo assoluto (1 singola unità) e mantengono un consumo di LUT e Flip-Flop estremamente ridotto. Sono le candidate ideali per dispositivi FPGA a bassissimo costo o alimentati a batteria.
- **Migliore per Prestazioni Assolute (High-Throughput):** La **Soluzione 10** (Total Unroll del MAC con memoria completamente partizionata) stravince nel dominio temporale. Costringendo il sintetizzatore a istanziare 11 moltiplicatori paralleli, la latenza crolla a soli 4 cicli di clock. Valutata sotto lo stress test dell'**Operation Chaining** (con vincolo rilassato), ha dimostrato di poter abbattere il **Delay** totale a circa 21.17 ns. È la scelta obbligata per acceleratori in ambito data-center o elaborazione DSP in tempo reale ad altissima frequenza.
- **Miglior Compromesso (Area-Delay Product Ottimale):** La **Soluzione 09** (Loop Fission + Unroll dello shift + Pipeline del MAC) si posiziona esattamente sul «ginocchio» della curva di Pareto. Sfruttando la sovrapposizione temporale della pipeline unitamente a una memoria sbloccata dall'**Array Partitioning**, riesce ad abbattere la latenza a soli 16 cicli mantenendo l'impiego di blocchi **DSP** pari a 1. Questa architettura massimizza il riuso temporale delle risorse aritmetiche, offrendo un eccellente throughput con una frazione dell'area richiesta dalla soluzione srotolata totalmente.

### 5.3. Conclusioni

L'intero flusso di lavoro ha fornito preziose indicazioni sul comportamento del sintetizzatore **Vivado HLS** e sull'efficacia della metodologia adottata.

In primo luogo, si evince chiaramente che il sintetizzatore **non è uno strumento magico**: applicare direttive di parallelismo (come **Unroll** o **Pipeline**) su strutture dati non predisposte (es. array allocati su singole BRAM a doppia porta) porta inevitabilmente a conflitti strutturali che degradano il **Critical Path** ed espandono inutilmente la logica di controllo. L'**Array Partitioning** si è confermato come la **enabling transformation** per eccellenza, senza la quale il parallelismo spaziale risulta inefficace.

Sotto il profilo metodologico, l'approccio di esplorazione incrementale si è rivelato vincente. L'applicazione preventiva della **Bitwidth Optimization** ha snellito le reti di routing prima di procedere con lo srotolamento, evitando l'esplosione combinatoria dell'area che si sarebbe verificata istanziando risorse parallele a 32 *bit*. Inoltre, lo **Stress Test** sul periodo di clock ha dimostrato la capacità di Vivado di eseguire un efficiente **Operation Chaining**, accorpando dinamicamente la logica combinatoria a patto che i cicli siano srotolati e liberi da dipendenze tra iterazioni (loop-carried dependencies).

In conclusione, la progettazione in **High-Level Synthesis** si configura come un continuo esercizio di bilanciamento. Non esiste un'architettura universalmente «migliore», ma uno spettro di soluzioni valide. Il compito del progettista moderno non è più scrivere minuziosamente macchine a stati in VHDL, bensì guidare il compilatore, attraverso la profonda comprensione dell'hardware sottostante, per centrare lo specifico punto di ottimo richiesto dalle specifiche di sistema.